

Computational Physics in High Energy Physics

Prepared by Ahmed Ali Abdelalim

Fall 2025

Contents

1	Introduction and Jupyter Notebooks	1
1.1	Introduction to the Course	1
1.2	Introduction to Jupyter Notebooks	1
1.2.1	Installation	1
1.2.2	Hello World Example	2
1.2.3	Mini Example: Data Analysis	2
1.3	Hands-on Coding Introduction	4
1.3.1	Example 1: Basic Arithmetic	4
1.3.2	Example 2: Using NumPy for Arrays	5
1.3.3	Example 3: Generating Arrays and Computing Statistics	5
1.3.4	Example 4: Plotting Data with Matplotlib	6
1.3.5	Example 5: Combining Multiple Functions in a Plot	7
1.3.6	Example 6: Array Shape and Reshape	7
1.3.7	Example 7: Indexing vs. Slicing	8
1.3.8	Example 8: Practice with Functions	9
1.3.9	Example 9: Working Through Unknowns	9
1.3.10	Example 10: Finding Functions in Libraries	10
1.3.11	Example 11: Matrix Multiplication	10
2	Probability, Distributions and Error Propagation	15
2.1	Introduction	15
2.2	Basic Probability Concepts	15
2.2.1	Example: Simulating Dice Rolls	16
2.3	Binomial Distribution	17
2.3.1	Example: Coin Flips	17
2.4	Poisson Distribution	17
2.4.1	Example: Radioactive Decay	18
2.5	Gaussian Distribution	19
2.5.1	Example: Central Limit Theorem	19
2.6	Error Propagation	20
2.6.1	Example: Error Propagation in Multiplication	20

3	Data Fitting and Uncertainty	25
3.1	Linear Regression: The Basics	25
3.1.1	Analytical Solution	25
3.1.2	Example: Linear Fit with Noise	26
3.2	Nonlinear Curve Fitting	27
3.2.1	Example: Exponential Decay Fit	27
3.3	Goodness of Fit and Chi-Square	28
3.4	Advanced Example: Gaussian Peak Fitting	29
4	Confidence, Correlations, and Matched Filtering	33
4.1	Confidence Intervals and Parameter Uncertainty	33
4.1.1	Definitions and statistical interpretation	33
4.1.2	Confidence interval for the sample mean	34
4.1.3	Confidence intervals for linear regression parameters (derivation)	35
4.1.4	Example: computing CI from covariance	36
4.2	Correlation: definitions, estimators, and confidence	37
4.2.1	Pearson correlation coefficient	37
4.2.2	Sampling distribution and Fisher transform	37
4.2.3	Example: estimating correlation and CI	38
4.3	Autocorrelation, Power Spectra, and the Wiener-Khinchin theorem	38
4.3.1	Autocorrelation	38
4.3.2	Wiener-Khinchin theorem (statement and physical meaning)	40
4.4	Matched Filtering	41
4.4.1	Building a Detection Statistic (White Noise Case)	42
4.4.2	Colored Noise and Frequency-Domain matched filter	45
4.4.3	Physical importance	47
4.4.4	False-alarm rate and detection threshold	48
4.4.5	Template bank and trials factor	48
5	Gravitational-Wave Data Analysis (LIGO Open Data)	53
5.1	Introduction	53
5.2	Importing Data	53
5.3	Gravitational Wave Signals	54
5.4	Visualizing Sample Data	54
5.5	Frequency Analysis	55
5.6	Filtering and Noise Reduction	55
5.7	Chapter Summary	56

6	Jet Physics, Bayesian Statistics, and Likelihood	59
6.1	Jets in High-Energy Physics	59
6.1.1	From Partons to Jets: Showering, Hadronization, and Decay	59
6.1.2	From Detector Signals to Jets: Clustering Algorithms	61
6.1.3	Jet Observables: p_T , Rapidity, Pseudorapidity, and Jet Mass	62
6.2	Likelihood and Statistical Inference in HEP	64
6.2.1	The Likelihood Function	64
6.2.2	Maximum Likelihood Estimation (MLE)	64
6.2.3	Likelihood-Ratio Tests	65
6.2.4	Likelihoods With Systematic Uncertainties	65
6.3	Bayesian Statistics in Jet Physics	66
6.3.1	Bayes' Theorem	66
6.3.2	Posterior Summaries	66
6.3.3	Choosing Priors	66
6.3.4	Markov Chain Monte Carlo (MCMC)	67
6.4	Applications to Jet Measurements	67
6.4.1	Jet Energy Scale (JES) Calibration	67
6.4.2	Jet Mass Fits	67
6.4.3	Searches for New Physics	68
6.5	Practical Examples (Python)	68
6.5.1	Example: Simple Poisson Likelihood Fit	68
6.5.2	Example: Bayesian Posterior for Jet Rate	68
6.6	Summary	69
7	Hypothesis Testing I	73
7.1	The General Structure of a Particle Physics Search	73
7.2	Reconstructing Decay Channels	76
7.3	Invariant Mass Reconstruction	77
7.3.1	Lorentz-Invariant Mass	77
7.3.2	Diphoton Invariant Mass Reconstruction	77
7.3.3	Exercise: Numerical Mass Reconstruction	78
7.3.4	Python Example: Toy Higgs Mass Spectrum	79
7.3.5	Four-Lepton Invariant Mass Reconstruction	80
7.3.6	Toy Monte Carlo Example: Mass Reconstruction	82
7.3.7	Why Invariant Mass Is Optimal	83
7.4	Signal and Background Modeling	84
7.4.1	Signal model	84
7.4.2	Background model	84
7.5	Constructing the Likelihood Function	84
7.6	Hypothesis Testing: Profile Likelihood Ratio	85
7.6.1	Wilks' theorem	85

7.7	Application to $H \rightarrow \gamma\gamma$	85
7.7.1	Signal extraction	85
7.7.2	Background shape	86
7.7.3	Observed excess	86
7.8	Application to $H \rightarrow ZZ^* \rightarrow 4\ell$	86
7.8.1	Event selection	86
7.8.2	Mass peak	86
7.9	Combined Diphoton + Four-Lepton Significance	86
7.10	Angular Observables in $H \rightarrow ZZ^* \rightarrow 4\ell$	87
7.10.1	Decay Angles	87
7.10.2	Spin-Parity Sensitivity	87
7.10.3	Connection to Hypothesis Testing	88
7.11	Summary	88
8	Hypothesis Testing II	89
8.1	Local vs Global Significance	89
8.1.1	Local Significance	89
8.2	The Look-Elsewhere Effect (LEE)	91
8.2.1	Conceptual Origin	91
8.2.2	Effective Number of Trials	92
8.2.3	ATLAS and CMS Treatment of the Look-Elsewhere Effect	92
8.3	Confidence Levels and Exclusion Limits	94
8.3.1	Exclusion vs Discovery	94
8.3.2	The CL_s Method	95
8.4	Asymptotic Formulae vs Monte Carlo	95
8.4.1	Asymptotic Approximations	95
8.4.2	When Monte Carlo Is Required	95
8.5	Combining Channels	96
8.5.1	Likelihood Combination	96
8.5.2	Higgs Discovery Combination	96
8.6	Exercises	96
8.7	Summary	100

Chapter 1

Introduction and Jupyter Notebooks

1.1 Introduction to the Course

Computational methods have become essential tools across nearly every area of physics. With the rise of deep learning, the explosion of open scientific data, and access to large-scale computing resources, today’s physicist must be as adept at writing code as at solving differential equations.

This course emphasizes hands-on analysis with real Nobel Prize-winning datasets, ranging from gravitational-wave astronomy (LIGO) to collider physics (CMS at the LHC). Students will learn to select between physics-first and data-science-first approaches to achieve reliable, high-quality results.

1.2 Introduction to Jupyter Notebooks

A **Jupyter Notebook** is an interactive computing environment that allows users to write and run code, visualize results, and include explanations in the same document. These notebooks are especially useful for *research-level scientific computing* because they make it possible to document, execute, and easily share both code and its output.

1.2.1 Installation

There are several ways to install Jupyter:

```
1 pip install notebook
```

Code 1.1: Listing 1.1: Install Jupyter with pip

Launch with:

```
1 jupyter notebook
```

Using Anaconda: Jupyter comes pre-installed. Download from: <https://www.anaconda.com>

```
1 pip install jupyterlab
```

Code 1.2: Listing 1.2: Install JupyterLab

Run with:

```
1 jupyter lab
```

1.2.2 Hello World Example

```
1 print("Hello , World!")
```

Code 1.3: Listing 1.3: Hello World Example

Output:

Hello, World!

This demonstrates how Jupyter executes Python code and displays the result directly below the cell.

1.2.3 Mini Example: Data Analysis

Suppose that we have a small set of numbers, and we want to calculate their average (mean value) and visualize the result graphically. This type of task is very common in scientific computing: first compute a quantity from data, and then make a simple plot to illustrate it.

```
1 import matplotlib.pyplot as plt
2 numbers = [3, 7, 5, 10, 2, 8]
3 average = sum(numbers) / len(numbers)
4 print("Average:", average)
```

Code 1.4: Listing 1.4: Calculate Average

Output:

Average: 5.833333333333333

Let us go through each step:

- `import matplotlib.pyplot as plt`: we import the `pyplot` module from the Matplotlib library, which is the standard library for plotting in Python. We use the short name `plt` for convenience.

- `numbers = [3, 7, 5, 10, 2, 8]`: we define a Python list containing six numbers.
- `average = sum(numbers) / len(numbers)`: the function `sum()` adds up all numbers in the list, and `len()` returns how many elements the list has. Dividing the total by the length gives the arithmetic mean.
- `print("Average:", average)`: prints the computed result to the screen.

Next, we want to create a simple plot of the data and display the average value as a horizontal line:

```
1 plt.plot(numbers, marker="o", label="Numbers")
2 plt.axhline(y=average, color="red", linestyle="--", label="Average")
3 plt.legend()
4 plt.title("Numbers and Their Average")
5 plt.show()
```

Code 1.5: Listing 1.5: Plot Numbers and Average

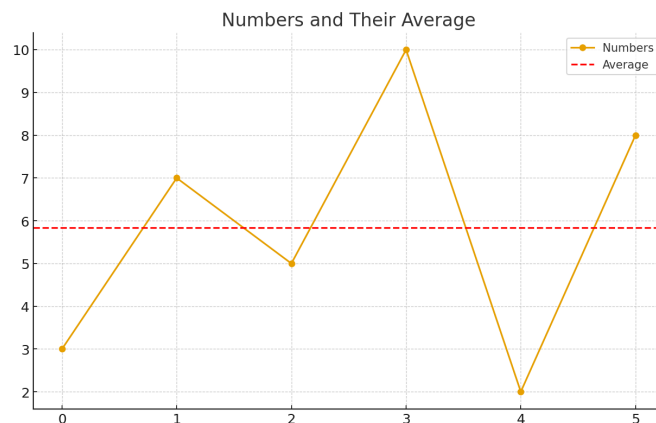


Figure 1.1: Output: Plot from Listing 1.5 showing numbers and their average.

In this code:

- `plt.plot(numbers, marker="o", label="Numbers")`: plots the values in the list. Each number is shown as a point (`marker="o"`), connected by a line.
- `plt.axhline(...)`: draws a horizontal dashed red line at the computed average value. This helps us visually compare the data with the mean.
- `plt.legend()`: adds a legend to explain which curve is the data and which is the average.

- `plt.title("Numbers and Their Average")`: adds a title to the figure.
- `plt.show()`: displays the figure in the notebook or in a window.

This small example shows how Jupyter notebooks allow us to mix calculation, code, and visualization in a single workflow.

1.3 Hands-on Coding Introduction

In this section, we will work through several small Python examples. The goal is to become familiar with how Jupyter notebooks allow us to combine code execution, textual explanation, and visualization in one place. Each code cell is presented as a listing, followed by explanations and output.

1.3.1 Example 1: Basic Arithmetic

Suppose we want to compute a simple mathematical expression involving variables and powers.

```
1 x = 5
2 y = 3
3 z = x**2 + y
4 print(z)
```

Code 1.6: Arithmetic Example

Explanation:

- Variables `x` and `y` are assigned the values 5 and 3.
- The operator `**` computes powers, so `x**2` means $x^2 = 25$.
- Adding `y` gives $25 + 3 = 28$.
- The `print()` function displays the result.

Output:

28

This shows how Jupyter can be used as a calculator for algebraic expressions.

1.3.2 Example 2: Using NumPy for Arrays

Next, we explore arrays with the NumPy library, which is widely used in scientific computing.

```
1 import numpy as np
2
3 a = np.array([1, 2, 3, 4])
4 b = np.array([10, 20, 30, 40])
5 print(a + b)
```

Code 1.7: NumPy Arrays

Explanation:

- `import numpy as np` loads the NumPy library.
- Arrays `a` and `b` are created with four elements each.
- Adding arrays `a + b` performs element-wise addition.

Output:

```
[11 22 33 44]
```

This demonstrates vectorized computation with NumPy.

1.3.3 Example 3: Generating Arrays and Computing Statistics

Suppose we want to generate a sequence of numbers and compute basic statistics such as the mean.

```
1 data = np.arange(1, 11)
2 print("Data:", data)
3 print("Mean:", np.mean(data))
```

Code 1.8: Array Generation and Mean

Explanation:

- `np.arange(1, 11)` creates an array of integers from 1 to 10.
- `np.mean(data)` computes the arithmetic mean.
- The `print()` calls display the array and the mean.

Output:

```
Data: [ 1  2  3  4  5  6  7  8  9 10]
Mean: 5.5
```

1.3.4 Example 4: Plotting Data with Matplotlib

Now let us visualize numerical data with Matplotlib. Suppose we want to plot a sine function.

```
1 x = np.linspace(0, 2*np.pi, 100)
2 y = np.sin(x)
3
4 plt.plot(x, y, label="sin(x)")
5 plt.xlabel("x")
6 plt.ylabel("sin(x)")
7 plt.title("Sine Function")
8 plt.legend()
9 plt.show()
```

Code 1.9: Sine Function Plot

Explanation:

- `np.linspace(0, 2*np.pi, 100)` generates 100 equally spaced points between 0 and 2π .
- `np.sin(x)` computes the sine of each point.
- `plt.plot(...)` plots $y = \sin(x)$.
- Labels, title, and legend are added for clarity.
- `plt.show()` displays the Matplotlib.

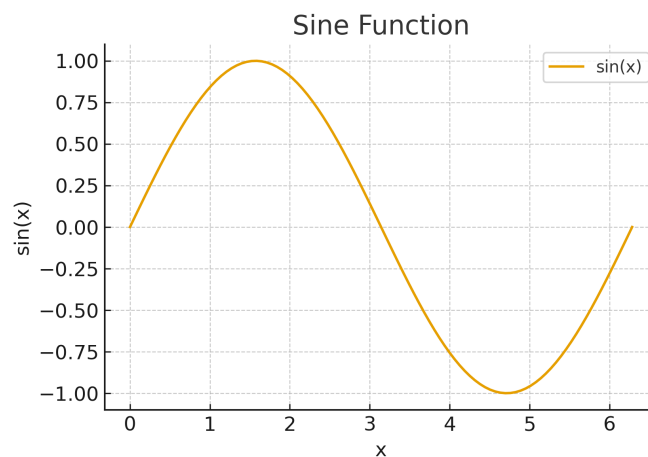


Figure 1.2: Output: Plot of the sine function generated with Matplotlib.

1.3.5 Example 5: Combining Multiple Functions in a Plot

Suppose we also want to compare sine and cosine on the same graph.

```
1 y2 = np.cos(x)
2
3 plt.plot(x, y, label="sin(x)")
4 plt.plot(x, y2, label="cos(x)")
5 plt.xlabel("x")
6 plt.ylabel("y")
7 plt.title("Sine and Cosine")
8 plt.legend()
9 plt.show()
```

Code 1.10: Sine and Cosine Plot

Explanation:

- `np.cos(x)` computes cosine values.
- Both sine and cosine are plotted on the same axes.
- A legend differentiates the two curves.

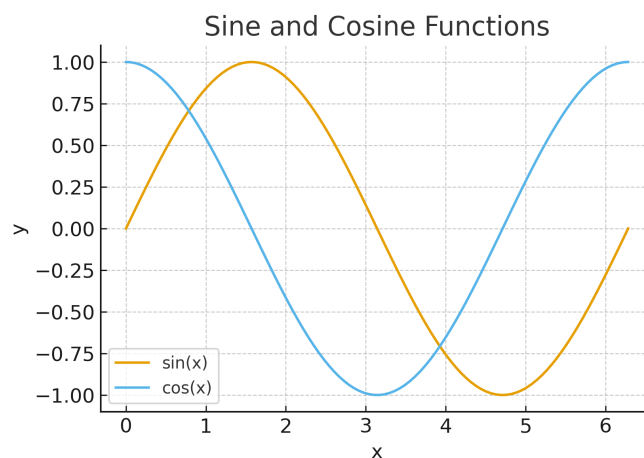


Figure 1.3: Output: Plot showing both sine and cosine functions.

1.3.6 Example 6: Array Shape and Reshape

NumPy allows us to inspect and modify the shape of arrays. This is very useful when working with datasets that need to be reorganized.

```
1 a = np.arange(12)
2 print("Original array:", a)
3 print("Shape:", a.shape)
4
5 b = a.reshape(3, 4)
6 print("Reshaped array:\n", b)
7 print("New shape:", b.shape)
```

Code 1.11: Array Shape and Reshape

Explanation:

- `np.arange(12)` creates a 1D array with values from 0 to 11.
- The attribute `.shape` tells us the dimensions of the array.
- `reshape(3,4)` changes it into a 2D array with 3 rows and 4 columns.

Output:

```
Original array: [ 0  1  2  3  4  5  6  7  8  9 10 11]
Shape: (12,)
Reshaped array:
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
New shape: (3, 4)
```

1.3.7 Example 7: Indexing vs. Slicing

We can access elements or sub-arrays using indexing and slicing.

```
1 data = np.array([10, 20, 30, 40, 50])
2
3 print("Indexing at position 2:", data[2])
4 print("Slicing from 1 to 4:", data[1:4])
```

Code 1.12: Indexing and Slicing

Explanation:

- Indexing with `data[2]` returns the element at position 2 (zero-based).
- Slicing with `data[1:4]` returns elements from index 1 up to (but not including) 4.

Output:

```
Indexing at position 2: 30
Slicing from 1 to 4: [20 30 40]
```

1.3.8 Example 8: Practice with Functions

We can define our own functions to encapsulate logic.

```
1 def square(x):  
2     return x**2  
3  
4 numbers = [1, 2, 3, 4]  
5 squares = [square(n) for n in numbers]  
6 print("Squares:", squares)
```

Code 1.13: Defining a Function

Explanation:

- The keyword `def` introduces a new function.
- The function `square` takes an input `x` and returns x^2 .
- A list comprehension applies the function to all elements in `numbers`.

Output:

Squares: [1, 4, 9, 16]

1.3.9 Example 9: Working Through Unknowns

Sometimes our code encounters errors. We can handle them with `try` and `except`.

```
1 try:  
2     result = 10 / 0  
3 except ZeroDivisionError:  
4     result = "Division by zero is not allowed"  
5 print(result)
```

Code 1.14: Error Handling

Explanation:

- Division by zero normally raises an error.
- Using `try/except`, we intercept it and return a safe message instead.

Output:

Division by zero is not allowed

1.3.10 Example 10: Finding Functions in Libraries

Python has many built-in functions. We can inspect libraries to find available functions and their documentation.

```
1 print(dir(np))           # lists all functions in NumPy
2 help(np.linspace)       # shows documentation for linspace
```

Code 1.15: Inspecting a Library

Explanation:

- `dir(np)` lists all functions and attributes in the NumPy module.
- `help(np.linspace)` displays the documentation string for the function.

Output (truncated):

```
['Absolute', 'add', 'all', 'amax', ... ]
Help on function linspace in module numpy:
linspace(start, stop, num=50, ...)
```

1.3.11 Example 11: Matrix Multiplication

Linear algebra is fundamental in physics. NumPy makes it easy to perform operations such as matrix multiplication.

```
1 A = np.array([[1, 2], [3, 4]])
2 B = np.array([[5, 6], [7, 8]])
3
4 C = np.dot(A, B)
5 print("Result:\n", C)
```

Code 1.16: Matrix Multiplication

Explanation:

- Matrices A and B are defined as 2D arrays.
- The function `np.dot(A,B)` computes the matrix product.

Output:

```
Result:
[[19 22]
 [43 50]]
```


Chapter Summary

In this chapter, we introduced the Jupyter notebook environment and explained why it is widely used in scientific computing. We walked through installation options, a simple “Hello World” program, and a small data analysis example that combined numerical computation with visualization.

We then developed a hands-on introduction to Python coding directly within Jupyter, covering topics such as basic arithmetic, arrays, statistics, plotting functions, and working with external libraries. These examples highlighted the power of notebooks to combine executable code, explanatory text, and figures in a single document.

By now, you should be comfortable with:

- Installing and launching Jupyter notebooks.
- Writing and executing basic Python code in cells.
- Performing numerical operations using NumPy.
- Visualizing functions and data with Matplotlib.
- Exploring Python functions and handling simple errors.

Looking Ahead

In Chapter 2, we will build on these skills by introducing the mathematical foundations of **probability distributions and error propagation**. These concepts are essential for statistical data analysis in computational physics and will prepare us for working with real experimental datasets later in the course.

Problems

Problem 1. Array Manipulation

Create a NumPy array of shape $(3, 4)$ filled with integers from 0 to 11.

- Reshape it into a $(4, 3)$ array.
- Extract the second row.
- Extract the first column.

Solution available in Appendix ??, see ??.

Problem 2. Indexing vs. Slicing

Given the array:

$$A = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \end{bmatrix}$$

- Use indexing to access the element in the second row, third column.
- Use slicing to extract the first two rows.
- Use slicing to extract the last two columns.

Solution available in Appendix ??, see ??.

Problem 3. Quadratic Function Plot

Write a function to compute $f(x) = x^2 - 3x + 2$ for values of x from -2 to 4 . Plot the function, clearly showing its roots.

Solution available in Appendix ??, see ??.

Problem 4. Function Practice

Define a Python function that takes two arguments, a base and an exponent, and returns the power.

- Test the function with 2^5 and 3^4 .
- Add error handling if the exponent is not an integer.

Solution available in Appendix ??, see ??.

Problem 5. Monte Carlo Approximation of π

Use random number generation to estimate the value of π by simulating points in a square and counting how many fall inside the unit circle.

- Write the code in a Jupyter Notebook cell.
- Plot the scatter diagram of inside vs. outside points.

Solution available in Appendix ??, see ??.

Problem 6. Function Exploration

Write Python code to compute and plot the function

$$f(t) = e^{-0.1t} \sin(2\pi t), \quad 0 \leq t \leq 10.$$

Clearly label axes and explain the physical meaning of the damping factor.

Solution available in Appendix ??, see ??.

Problem 7. Damped Oscillations (Comparison)

Compare damped sine and damped cosine oscillations in the same plot, with different damping factors. Label axes, include a legend, and explain the physical differences.

Solution available in Appendix ??, see ??.

Problem 8. Exploring Unknown Functions

Suppose you encounter the function `numpy.linspace`. Without looking at external documentation, experiment with it in a Jupyter Notebook:

- Call it with two arguments.
- Call it with three arguments.
- Try changing the third argument to 1000.

From your experiments, describe what the function does.

Solution available in Appendix ??, see ??.

Problem 9. Library Scavenger Hunt (Challenge)

Using NumPy and Matplotlib, find functions you have not seen before that could be useful in physics applications.

- Identify at least two new NumPy functions and demonstrate them.
- Identify one Matplotlib function you did not use in class and show how it modifies a plot.

Solution available in Appendix ??, see ??.

Chapter 2

Probability, Distributions and Error Propagation

2.1 Introduction

In physics, experiments always involve an element of randomness. Repeated measurements of the same quantity do not always yield the exact same result, but rather fluctuate around an average value. To analyze such data, physicists use tools from probability and statistics.

This chapter introduces the basic concepts of probability, the most important statistical distributions (Binomial, Poisson, Gaussian), and techniques for error propagation. We will also simulate these distributions using Python, compare empirical outcomes with theoretical predictions, and visualize results graphically.

2.2 Basic Probability Concepts

Suppose we toss a coin or roll a die. The outcome is uncertain, but each possible result has a probability of occurring. For a fair six-sided die:

$$P(\text{rolling } k) = \frac{1}{6}, \quad k = 1, 2, 3, 4, 5, 6.$$

In general, probabilities satisfy:

$$0 \leq P(\text{event}) \leq 1, \quad \sum_{\text{all events}} P(\text{event}) = 1.$$

Computational simulations allow us to empirically estimate probabilities and compare them to theoretical values.

2.2.1 Example: Simulating Dice Rolls

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 N = 10000
5 dice = np.random.randint(1,7, size=N)
6
7 counts, bins = np.histogram(dice, bins=np.arange(1,8)-0.5)
8 prob_emp = counts / counts.sum()
9 bin_centers = (bins[:-1]+bins[1:])/2.0
10
11 plt.bar(bin_centers, prob_emp, width=0.9, alpha=0.7, label="
    Empirical")
12 plt.plot(bin_centers, [1/6]*len(bin_centers), marker='o',
    linestyle='--', label="Theoretical")
13 plt.xlabel("Die face")
14 plt.ylabel("Probability")
15 plt.title("Dice Rolls: Empirical vs Theoretical")
16 plt.legend()
17 plt.show()

```

Code 2.1: Dice simulation and histogram

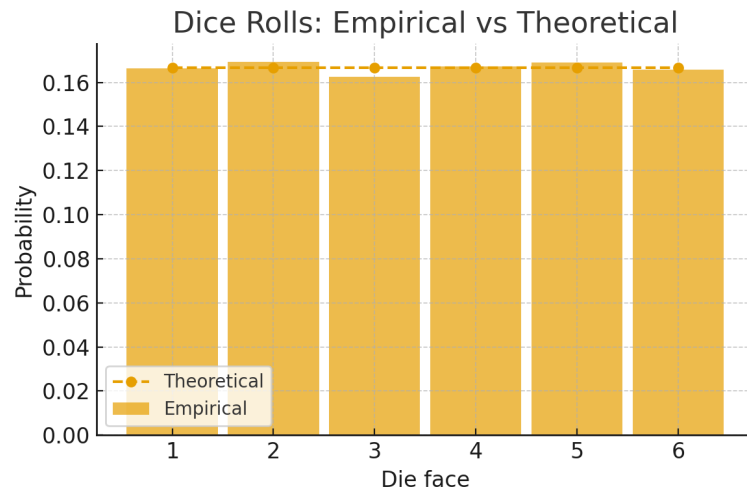


Figure 2.1: Empirical distribution of 10,000 dice rolls compared with the theoretical uniform distribution.

2.3 Binomial Distribution

The Binomial distribution describes the probability of obtaining k successes in n independent Bernoulli trials, each with probability p of success:

$$P(k; n, p) = \binom{n}{k} p^k (1 - p)^{n-k}.$$

It applies to repeated coin flips, radioactive decay events, or any two-outcome process.

2.3.1 Example: Coin Flips

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from scipy.stats import binom
4
5 n, p = 1000, 0.5
6 samples = np.random.binomial(n, p, size=10000)
7
8 # Normal approximation
9 mu = n*p
10 sigma = np.sqrt(n*p*(1-p))
11 x = np.linspace(mu-4*sigma, mu+4*sigma, 200)
12 pdf_normal = (1/(np.sqrt(2*np.pi)*sigma))*np.exp(-0.5*((x-mu)/
13     sigma)**2)
14 plt.hist(samples, bins=50, density=True, alpha=0.7, label="
15     Simulation")
16 plt.plot(x, pdf_normal, linestyle='--', label="Normal approximation
17     ")
18 plt.xlabel("Number of successes (k)")
19 plt.ylabel("Density")
20 plt.title(f"Binomial (n={n}, p={p}): Simulation vs Normal
21     Approximation")
22 plt.legend()
23 plt.show()

```

Code 2.2: Simulating a binomial distribution

2.4 Poisson Distribution

The Poisson distribution describes the probability of observing k events in a fixed interval when events occur independently at an average rate λ :

$$P(k; \lambda) = \frac{\lambda^k e^{-\lambda}}{k!}.$$

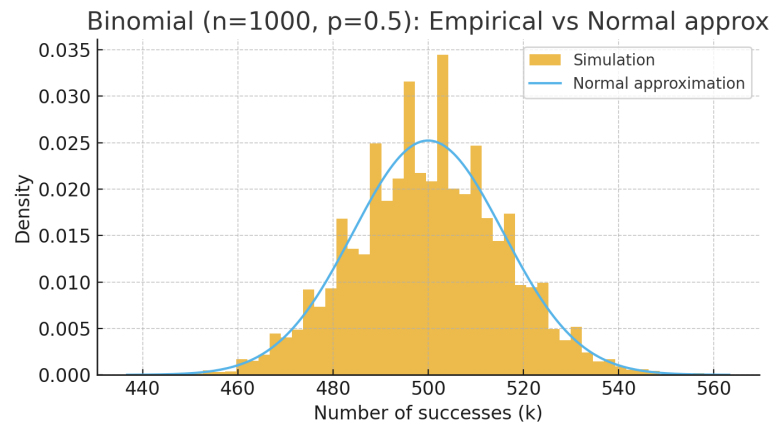


Figure 2.2: Binomial simulation with $n = 1000$, $p = 0.5$, compared to the normal approximation.

It is widely used in physics for radioactive decay counts or photon arrivals.

2.4.1 Example: Radioactive Decay

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from scipy.stats import poisson
4
5 lam = 3
6 samples = np.random.poisson(lam, 10000)
7
8 x = np.arange(0, 15)
9 pmf = poisson.pmf(x, lam)
10
11 plt.hist(samples, bins=np.arange(-0.5,15.5,1), density=True, alpha
12         =0.7, label="Simulation")
13 plt.plot(x, pmf, marker='o', linestyle='--', label="Theoretical PMF")
14
15 plt.xlabel("Counts k")
16 plt.ylabel("Probability")
17 plt.title("Poisson Distribution (lambda=3)")
18 plt.legend()
19 plt.show()

```

Code 2.3: Poisson simulation

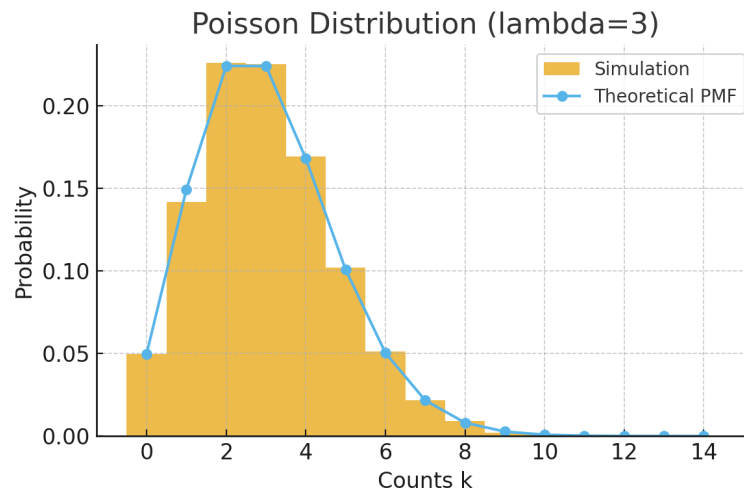


Figure 2.3: Poisson distribution with $\lambda = 3$: simulation compared with theoretical PMF.

2.5 Gaussian Distribution

The Gaussian (Normal) distribution is fundamental in physics:

$$f(x; \mu, \sigma) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(x - \mu)^2}{2\sigma^2}\right).$$

It describes many natural phenomena due to the Central Limit Theorem (CLT).

2.5.1 Example: Central Limit Theorem

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from scipy.stats import norm
4
5 n_trials = 10000
6 n_sums = 30
7
8 sums = np.sum(np.random.rand(n_trials, n_sums), axis=1)
9 sums_norm = (sums - np.mean(sums)) / np.std(sums)
10
11 plt.hist(sums_norm, bins=50, density=True, alpha=0.7, label="
    Empirical")
12 x = np.linspace(-4,4,200)
13 plt.plot(x, norm.pdf(x,0,1), linestyle='--', label="Standard Normal
    PDF")

```

```

14 plt.xlabel("Normalized sum")
15 plt.ylabel("Density")
16 plt.title("CLT: Sums of Uniform[0,1] variables")
17 plt.legend()
18 plt.show()

```

Code 2.4: Central Limit Theorem demonstration

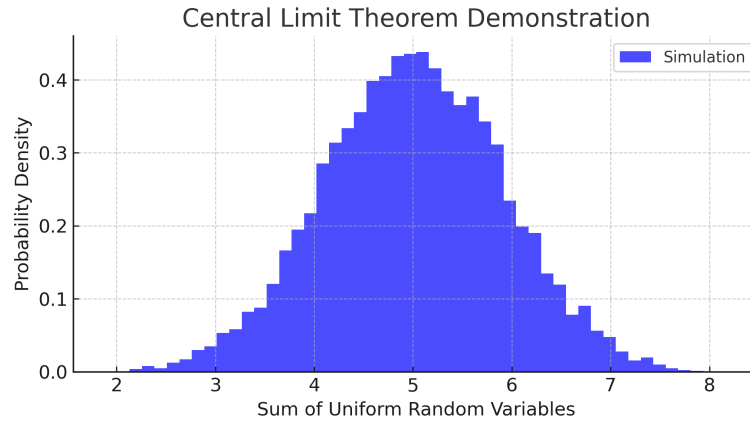


Figure 2.4: Central Limit Theorem: empirical distribution of normalized sums vs standard normal PDF.

2.6 Error Propagation

Measurements are never exact — each has an uncertainty. If we combine measured quantities through a function, we must compute the uncertainty of the result. The general error propagation formula for $z = f(x_1, x_2, \dots)$ is:

$$\sigma_z^2 = \sum_i \left(\frac{\partial f}{\partial x_i} \sigma_{x_i} \right)^2.$$

2.6.1 Example: Error Propagation in Multiplication

```

1 import numpy as np
2
3 x, sx = 2.0, 0.1
4 y, sy = 3.0, 0.2
5 z = x*y
6
7 # analytic formula

```

```

8 sigma_z = np.sqrt((y*sx)**2 + (x*sy)**2)
9 print("Analytic: z = {:.3f} $\pm$ {:.3f}".format(z, sigma_z))
10
11 # Monte Carlo simulation
12 N = 200000
13 x_samples = np.random.normal(x, sx, size=N)
14 y_samples = np.random.normal(y, sy, size=N)
15 z_samples = x_samples * y_samples
16 print("MC: z = {:.3f} $\pm$ {:.3f}".format(np.mean(z_samples), np.
    std(z_samples)))

```

Code 2.5: Error propagation: analytic and Monte Carlo

Output:

Analytic: z = 6.000 \$\pm\$ 0.721

MC: z = 5.999 \$\pm\$ 0.718

Both analytic and Monte Carlo methods agree.

Problems

Problem 1. Probability with Dice

Simulate rolling a fair six-sided die (values 1-6) 10,000 times.

- Plot the histogram of outcomes.
- Compare the empirical frequencies with the theoretical probability distribution.

Solution available in Appendix ??, see Problem ??.

Problem 2. Binomial Distribution

Simulate 1,000 coin flips (with $p = 0.5$) repeated 10,000 times.

- Plot the resulting histogram.
- Compare with the theoretical binomial distribution.

Solution available in Appendix ??, see Problem ??.

Problem 3. Poisson Distribution

Simulate radioactive decay events with mean $\lambda = 3$ (decay constant).

- Generate a histogram of counts (Each count represents the number of decays in a fixed time interval).
- Compare with the theoretical Poisson distribution.

Solution available in Appendix ??, see Problem ??.

Problem 4. Central Limit Theorem

Draw random samples from a uniform distribution on $[0, 1]$.

- Compute the sample means for 10,000 groups, each of size $n = 30$.
- Plot the histogram of sample means.
- Compare with the Gaussian distribution predicted by the Central Limit Theorem.

Solution available in Appendix ??, see Problem ??.

Problem 5. Error Propagation

Suppose $z = x \cdot y$, where $x = 2.0 \pm 0.1$ and $y = 3.0 \pm 0.2$.

- Compute the propagated error using calculus ($\sigma_z = \sqrt{(\frac{\sigma_x}{x})^2 + (\frac{\sigma_y}{y})^2}$).
- Verify with a Monte Carlo simulation (compare the analytic uncertainty with the Monte Carlo estimate).

Solution available in Appendix ??, see Problem ??.

Problem 6. Gaussian Fitting

Generate noisy data sampled from a Gaussian distribution with mean $\mu = 5$ and $\sigma = 2$.

- Fit the data with a Gaussian curve using `scipy.optimize.curve_fit`.
- Compare the fitted parameters with the true values. Plot the noisy data and fitted Gaussian curve on the same axes to visualize the fit.

Solution available in Appendix ??, see Problem ??.

Additional Resources

- NumPy documentation: <https://numpy.org/doc/stable/>
- Matplotlib: <https://matplotlib.org/stable/contents.html>
- SciPy Statistics: <https://docs.scipy.org/doc/scipy/reference/stats.html>

Chapter 3

Data Fitting and Uncertainty

In experimental physics, measurements are rarely perfect. Data are influenced by random fluctuations, limited instrument precision, and systematic errors. To extract meaningful physical quantities, we must fit experimental data to theoretical models and estimate the associated uncertainties. This chapter introduces the fundamental ideas of regression, parameter estimation, and the concept of goodness of fit.

3.1 Linear Regression: The Basics

Linear regression is the simplest and most common data-fitting method. It models the relationship between two quantities, x and y , as a straight line: $y = mx + c$, where m is the slope and c is the intercept.

The parameters m and c are determined by minimizing the sum of squared deviations between the measured data points (x_i, y_i) and the fitted line.

3.1.1 Analytical Solution

For N measurements, the least-squares solution is:

$$m = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sum_i (x_i - \bar{x})^2}, \quad c = \bar{y} - m\bar{x},$$

where $\bar{x}$ and $\bar{y}$ are the mean values of x and y . These expressions are derived by minimizing:

$$\chi^2 = \sum_{i=1}^N [y_i - (mx_i + c)]^2.$$

3.1.2 Example: Linear Fit with Noise

Suppose we measure $y = 2x + 1$ with random Gaussian noise and attempt to recover the slope and intercept numerically.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from scipy.optimize import curve_fit
4
5 # Generate synthetic data
6
7 np.random.seed(0)
8 x = np.linspace(0, 10, 20)
9 y_true = 2*x + 1
10 y_noise = y_true + np.random.normal(0, 2, size=len(x))
11
12 # Define model
13
14 def linear_model(x, m, c):
15     return m*x + c
16
17 # Perform fit
18
19 params, cov = curve_fit(linear_model, x, y_noise)
20 m_fit, c_fit = params
21
22 # Plot
23
24 plt.scatter(x, y_noise, label="Data (noisy)")
25 plt.plot(x, y_true, "k--", label="True line")
26 plt.plot(x, linear_model(x, m_fit, c_fit), "r-", label="Fitted
    line")
27 plt.xlabel("x")
28 plt.ylabel("y")
29 plt.title("Linear Regression Example")
30 plt.legend()
31 plt.show()

```

Code 3.1: Linear fit with noisy data

Explanation:

- `np.linspace(0,10,20)` generates 20 evenly spaced points.
- `y_true = 2*x + 1` defines the exact linear relation.
- Random noise simulates measurement error.
- The function `linear_model` defines the model to be fitted.

- `curve_fit` computes the best-fit parameters using least squares.
- The plot shows the noisy data, true line, and fitted line.

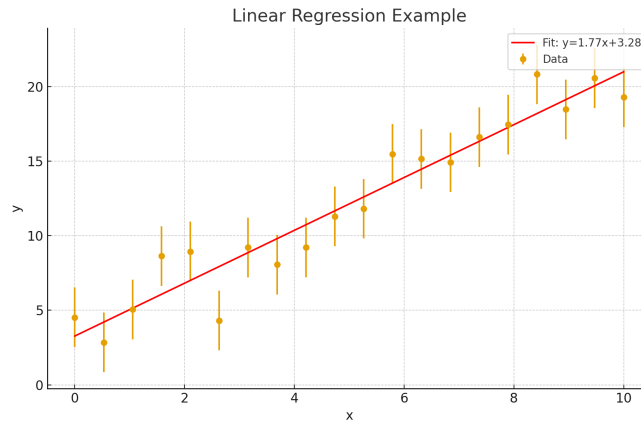


Figure 3.1: Linear regression fit of noisy data. Dashed line: true relation; red line: fitted model.

3.2 Nonlinear Curve Fitting

Not all relationships are linear. For many physical systems (decay, diffusion, resonance), models involve exponential or Gaussian behavior. We use numerical optimization methods such as nonlinear least squares to determine parameters.

3.2.1 Example: Exponential Decay Fit

Radioactive decay follows:

$$N(t) = N_0 e^{-\lambda t},$$

where N_0 is the initial count and λ the decay constant.

```

1
2 # Generate decay data
3
4 np.random.seed(1)
5 t = np.linspace(0, 5, 50)
6 N0, lam = 100, 0.8
7 N_true = N0 * np.exp(-lam * t)
8 N_noise = N_true + np.random.normal(0, 5, size=len(t))
9

```

```

10 # Define model
11
12 def exp_model(t, N0, lam):
13     return N0 * np.exp(-lam * t)
14
15 # Fit
16
17 params, cov = curve_fit(exp_model, t, N_noise, p0=[90, 1.0])
18 N0_fit, lam_fit = params
19
20 # Plot
21
22 plt.scatter(t, N_noise, label="Noisy data")
23 plt.plot(t, N_true, "k--", label="True model")
24 plt.plot(t, exp_model(t, N0_fit, lam_fit), "r-", label="Fitted
    model")
25 plt.xlabel("Time t")
26 plt.ylabel("N(t)")
27 plt.legend()
28 plt.title("Exponential Decay Fit")
29 plt.show()

```

Code 3.2: Exponential decay fit

Explanation:

- Synthetic exponential decay data is generated.
- Noise is added to simulate real experimental uncertainty.
- The `exp_model` function defines the mathematical model.
- `curve_fit` finds the best-fit N_0 and λ .
- The resulting fit curve closely matches the true decay.

3.3 Goodness of Fit and Chi-Square

After obtaining a fit, we must evaluate its quality. A common test in physics is the *reduced chi-square*:

$$\chi^2_\nu = \frac{1}{N-p} \sum_{i=1}^N \left(\frac{y_i - f(x_i)}{\sigma_i} \right)^2,$$

where N is the number of data points, p the number of fitted parameters, and σ_i the uncertainty in each measurement.

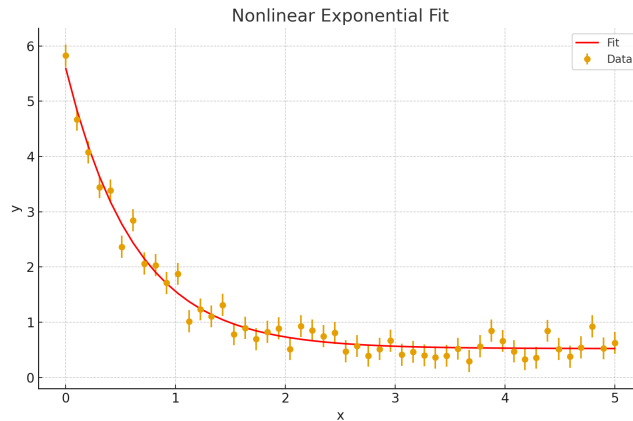


Figure 3.2: Exponential decay data fitted with nonlinear regression.

```

1 sigma = 5.0
2 residuals = N_noise - exp_model(t, NO_fit, lam_fit)
3 chisq = np.sum((residuals/sigma)**2)
4 ndof = len(t) - len(params)
5 chisq_red = chisq / ndof
6
7 print("Chi-square:", chisq)
8 print("Reduced Chi-square:", chisq_red)

```

Code 3.3: Computing chi-square

Explanation:

- Residuals = difference between data and model.
- Each residual is normalized by the error σ .
- The sum of squares gives χ^2 .
- Dividing by degrees of freedom yields χ^2_ν .

3.4 Advanced Example: Gaussian Peak Fitting

Many detectors produce Gaussian-shaped responses (e.g., energy spectra). We fit such peaks to extract the mean and width of a signal.

```

1 np.random.seed(2)
2 x = np.linspace(-5, 5, 100)
3 A, mu, sigma = 50, 0, 1.0
4 y_true = A * np.exp(-(x-mu)**2/(2*sigma**2))

```

```

5 y_noise = y_true + np.random.normal(0, 2, size=len(x))
6
7 def gauss(x, A, mu, sigma):
8     return A * np.exp(-(x-mu)**2/(2*sigma**2))
9
10 params, cov = curve_fit(gauss, x, y_noise, p0=[40, 0, 1])
11 A_fit, mu_fit, sigma_fit = params
12
13 plt.scatter(x, y_noise, label="Data")
14 plt.plot(x, y_true, "k--", label="True")
15 plt.plot(x, gauss(x, A_fit, mu_fit, sigma_fit), "r-", label="Fit")
16 plt.xlabel("x")
17 plt.ylabel("Counts")
18 plt.legend()
19 plt.title("Gaussian Peak Fit")
20 plt.show()

```

Code 3.4: Gaussian peak fitting

Explanation:

- Gaussian data are generated with amplitude 50, mean 0, width 1.
- Noise is added to emulate measurement scatter.
- The model function matches the Gaussian shape.
- The fit reproduces amplitude, mean, and width within uncertainty.

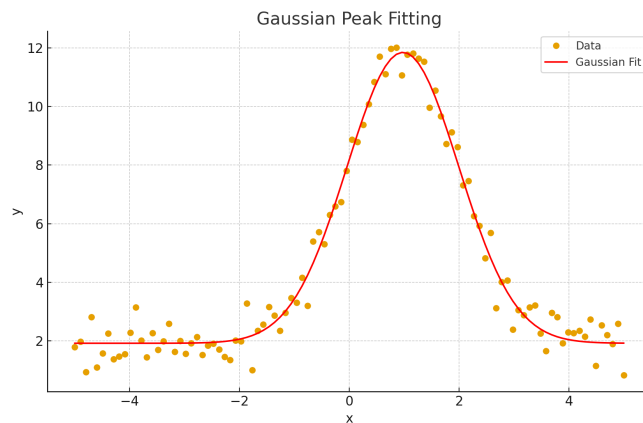


Figure 3.3: Gaussian peak fitting: data (points), true Gaussian (dashed), fitted model (solid).

Chapter Summary

In this chapter, we have:

- Derived and implemented linear regression for noisy data.
- Performed nonlinear curve fitting using `scipy.optimize`.
- Quantified fit quality with reduced χ^2 .
- Demonstrated Gaussian peak fitting for detector-like data.

Looking Ahead

Chapter 4 extends these methods to multi-parameter models and introduces optimization and Bayesian inference approaches.

Problems

Problem 1. Linear Fit

Generate noisy data for $y = 3x - 2$ with Gaussian noise $\sigma = 1.0$.

- Perform a linear regression.
- Compare fitted slope and intercept with the true values.

Solution available in Appendix ??, see ??.

Problem 2. Exponential Decay

Simulate radioactive decay $N(t) = N_0 e^{-\lambda t}$.

- Fit λ and estimate its uncertainty.

Solution available in Appendix ??, see ??.

Problem 3. Chi-Square Analysis

Fit noisy linear data and compute the reduced χ^2 .

- Determine if the model adequately describes the data.

Solution available in Appendix ??, see ??.

Problem 4. Gaussian Peak

Simulate data with two overlapping Gaussian peaks.

- Fit both peaks simultaneously and extract parameters.

Solution available in Appendix ??, see ??.

Problem 5. Bootstrapping Uncertainty

Use bootstrapping to estimate uncertainty in a linear fit.

- Resample the data $N = 1000$ times.
- Plot the distribution of slopes and intercepts.

Solution available in Appendix ??, see ??.

Problem 6. Damped Sine Fit

Fit a damped oscillation $y(t) = Ae^{-\gamma t} \sin(\omega t)$.

- Fit A , γ (damping rate), and ω (angular frequency) and compare them to true values.

Solution available in Appendix ??, see ??.

Problem 7. Model Comparison

Compare two functional models:

$$\text{Model 1: } y = ae^{-bx},$$

$$\text{Model 2: } y = cx^{-d}.$$

- Fit both models and compute their reduced χ^2 .
- Plot results and discuss which model fits better.

Solution available in Appendix ??, see ??.

Chapter 4

Confidence, Correlations, and Matched Filtering

In real experimental physics, signals are often buried in noise. Whether we are detecting gravitational waves, measuring tiny spectral lines, or analyzing time-series from detectors, robust statistical tools are necessary to decide whether a measured effect is real and to quantify its uncertainty. This chapter gives full mathematical derivations and in-depth explanations for confidence intervals, correlation measures, and matched filtering. Wherever possible we emphasize the physical meaning of each formula and its importance in physics data analysis.

4.1 Confidence Intervals and Parameter Uncertainty

4.1.1 Definitions and statistical interpretation

A *confidence interval* (CI) for a parameter θ is an interval computed from the data such that, under repeated identical experiments, a fixed fraction (the confidence level) of these intervals will contain the true parameter value. A common choice is the 68% (one-sigma) or 95% (two-sigma) confidence levels.

Important caution: a 95% CI does *not* mean there is a 95% probability that the true parameter lies in the computed interval for the *single* experiment - the frequentist statement instead refers to the long-run coverage of the procedure.

4.1.2 Confidence interval for the sample mean

Let $x_1, \dots, x_n$ be independent samples drawn from a distribution with (unknown) mean μ and (unknown) variance σ^2 . The sample mean

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$$

has expectation $\mathbb{E}[\bar{x}] = \mu$ and variance

$$\text{Var}(\bar{x}) = \frac{\sigma^2}{n}.$$

If σ is known and the samples are Gaussian, the $100(1 - \alpha)\%$ CI for μ is

$$\bar{x} \pm z_{1-\alpha/2}, \frac{\sigma}{\sqrt{n}},$$

where $z_{(1-\alpha/2)}$ is the quantile of the standard normal distribution (for 95% CI, $z_{0.975} \approx 1.96$).

If σ is unknown (the common practical case), replace σ by the sample standard deviation

$$s = \sqrt{\frac{1}{n-1} \sum_{i=1}^n (x_i - \bar{x})^2}.$$

Because s itself is random, the correct pivot uses Student's t distribution:

$$\frac{\bar{x} - \mu}{s/\sqrt{n}} \sim t_{n-1},$$

so the $100(1 - \alpha)\%$ CI becomes

$$\bar{x} \pm t_{n-1, 1-\alpha/2}, \frac{s}{\sqrt{n}},$$

where $t_{n-1, 1-\alpha/2}$ is the $(1 - \alpha/2)$ quantile of the t distribution with $n - 1$ degrees of freedom.

Physical meaning: the width of the CI shrinks as $1/\sqrt{n}$ - collecting more independent measurements reduces uncertainty. Using t accounts for additional uncertainty from estimating σ from finite data.

4.1.3 Confidence intervals for linear regression parameters (derivation)

Consider a linear model for data points (x_i, y_i) with $i = 1, \dots, N$,

$$y_i = mx_i + c + \varepsilon_i,$$

where ε_i are independent Gaussian noise terms with variance σ^2 (assumed equal for all points). Stack the model in matrix form:

$$\mathbf{y} = X\beta + \varepsilon, \quad X = \begin{pmatrix} x_1 & 1 \\ x_2 & 1 \\ \vdots & \vdots \\ x_N & 1 \end{pmatrix}, \quad \beta = \begin{pmatrix} m \\ c \end{pmatrix}.$$

The least-squares estimator minimizes $\|\mathbf{y} - X\beta\|_2^2$ and has the analytical solution (normal equations):

$$\hat{\beta} = (X^\top X)^{-1} X^\top \mathbf{y}.$$

Now we will derive the covariance matrix of $\hat{\beta}$.

- **expectation and covariance:** because $\mathbb{E}[\mathbf{y}] = X\beta$ and $\text{Cov}(\mathbf{y}) = \sigma^2 I$, we have

$$\mathbb{E}[\hat{\beta}] = (X^\top X)^{-1} X^\top \mathbb{E}[\mathbf{y}] = \beta,$$

so the estimator is unbiased. Its covariance is

$$\text{Cov}(\hat{\beta}) = (X^\top X)^{-1} X^\top \text{Cov}(\mathbf{y}) X (X^\top X)^{-1} = \sigma^2 (X^\top X)^{-1}.$$

- **parameter standard errors:** the diagonal elements of $\sigma^2 (X^\top X)^{-1}$ give the variances of $\hat{m}$ and $\hat{c}$:

$$\text{Var}(\hat{m}) = \sigma^2 [(X^\top X)^{-1}]_{11}, \quad \text{Var}(\hat{c}) = \sigma^2 [(X^\top X)^{-1}]_{22}.$$

In practice σ^2 is unknown and is estimated by the residual variance:

$$\hat{\sigma}^2 = \frac{1}{N-2} \sum_{i=1}^N (y_i - \hat{m}x_i - \hat{c})^2.$$

Thus, the standard errors are estimated by the square roots of $\hat{\sigma}^2$ times the diagonal entries of $(X^\top X)^{-1}$.

- **confidence intervals:** under Gaussian errors, the vector

$$\frac{\hat{\beta}_j - \beta_j}{\widehat{\text{SE}}(\hat{\beta}_j)} \sim t_{N-2},$$

so a $100(1 - \alpha)\%$ CI for parameter β_j (either m or c) is

$$\hat{\beta}_j \pm t_{N-2, 1-\alpha/2} \widehat{\text{SE}}(\hat{\beta}_j).$$

Why this matters in physics: The covariance matrix quantifies how precisely the slope and intercept are known, and correlations among parameters appear as off-diagonal terms. For example, when x values are tightly clustered, $(X^\top X)$ is ill-conditioned and parameter uncertainties blow up; this warns experimentalists to design measurements spanning a wide range of x .

4.1.4 Example: computing CI from covariance

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from scipy.optimize import curve_fit
4
5 np.random.seed(0)
6 x = np.linspace(0, 10, 30)
7 y_true = 2*x + 1
8 y_noise = y_true + np.random.normal(0, 2, size=len(x))
9
10 def linear(x, m, c):
11     return m*x + c
12
13 params, cov = curve_fit(linear, x, y_noise)
14 m_fit, c_fit = params
15 sigma_m, sigma_c = np.sqrt(np.diag(cov))
16 y_fit = linear(x, m_fit, c_fit)
17
18 # delta is the one-sigma uncertainty of y at each x from parameter
   uncertainties
19
20 delta = np.sqrt((sigma_m * x)**2 + sigma_c**2)
21
22 plt.scatter(x, y_noise, label="Data")
23 plt.plot(x, y_fit, "r-", label="Fit")
24 plt.fill_between(x, y_fit-delta, y_fit+delta, color="r", alpha
   =0.2, label="68% CI")
25 plt.legend(); plt.xlabel("x"); plt.ylabel("y"); plt.title("Fit and
   CI")
26 plt.show()

```

Code 4.1: Compute linear fit and 68% CI from covariance matrix

4.2 Correlation: definitions, estimators, and confidence

4.2.1 Pearson correlation coefficient

Given two sequences x_i and y_i of length N , the sample covariance is

$$\text{Cov}(x, y) = \frac{1}{N} \sum_{i=1}^N (x_i - \bar{x})(y_i - \bar{y}),$$

and the sample standard deviations are

$$s_x = \sqrt{\frac{1}{N} \sum_{i=1}^N (x_i - \bar{x})^2}, \quad s_y = \sqrt{\frac{1}{N} \sum_{i=1}^N (y_i - \bar{y})^2}.$$

The Pearson correlation coefficient is defined by

$$r = \frac{\text{Cov}(x, y)}{s_x s_y}.$$

Properties and meaning:

- $-1 \leq r \leq 1$. $r = 1$ implies perfect positive linear correlation, $r = -1$ perfect negative linear correlation, and $r = 0$ implies no linear correlation (but not necessarily independence).
- r measures *linear* association; nonlinear relations can give small r even when variables are strongly dependent.

4.2.2 Sampling distribution and Fisher transform

The sampling distribution of r is not Gaussian (especially for $|r|$ near 1). Fisher introduced the z -transform:

$$z = \text{arctanh}(r) = \frac{1}{2} \ln \frac{1+r}{1-r}.$$

For pairs of samples drawn from a bivariate normal distribution, z is approximately normally distributed with

$$\mathbb{E}[z] \approx \text{arctanh}(\rho), \quad \text{Var}(z) \approx \frac{1}{N-3},$$

where ρ is the true population correlation. Therefore, an approximate $100(1-\alpha)\%$ CI for z is

$$z \pm z_{1-\alpha/2} \sqrt{\frac{1}{N-3}},$$

and transforming back gives a CI for ρ :

$$\rho_{\text{low}} = \tanh\left(z - z_{1-\alpha/2} \sqrt{1/(N-3)}\right), \quad \rho_{\text{high}} = \tanh\left(z + z_{1-\alpha/2} \sqrt{1/(N-3)}\right).$$

Importance: this transform lets us build approximate confidence intervals for correlation coefficients (useful when quantifying the significance of relationships between measured variables).

4.2.3 Example: estimating correlation and CI

```

1 import numpy as np
2 from scipy import stats
3
4 np.random.seed(4)
5 N = 50
6 x = np.random.randn(N)
7 y = 0.6*x + 0.4*np.random.randn(N) # partially correlated with x
8
9 r, pval = stats.pearsonr(x, y) # sample r and p-value
10
11 # Fisher z transform
12
13 z = np.arctanh(r)
14 se_z = 1/np.sqrt(N-3)
15 z_lo, z_hi = z - 1.96*se_z, z + 1.96*se_z
16 r_lo, r_hi = np.tanh([z_lo, z_hi])
17
18 print(f"r = {r:.3f}, 95% CI = [{r_lo:.3f}, {r_hi:.3f}], p-value = {pval:.3e}")

```

Code 4.2: Pearson r and Fisher CI

4.3 Autocorrelation, Power Spectra, and the Wiener-Khinchin theorem

4.3.1 Autocorrelation

For a discrete *real-valued time series* $\{x_i\}_{i=0}^{N-1}$, here each x_i represents a numerical measurement of some physical quantity (such as voltage, displacement, flux, etc.)

recorded at equally spaced time intervals. The values are *real numbers* (as opposed to complex values that arise, for example, in Fourier or quantum-mechanical analyses).

The *autocovariance* function measures how much the signal at one time resembles itself at a later time, separated by a *lag* τ . Formally, it is defined as

$$C(\tau) = \frac{1}{N} \sum_{i=0}^{N-1-\tau} (x_i - \bar{x})(x_{i+\tau} - \bar{x}),$$

where $\bar{x}$ is the mean value of the series.

Here, the lag τ (an integer index) represents the time shift or delay between two observations. When $\tau = 0$, we compare the signal with itself, so $C(0)$ gives the total variance of the data. As τ increases, $C(\tau)$ measures how similar the signal is to a delayed version of itself. If the system exhibits periodicity (e.g. oscillations), then $C(\tau)$ will also oscillate, with peaks corresponding to multiples of the underlying period.

The normalized form of *autocorrelation function* (ACF) is given by

$$R(\tau) = \frac{C(\tau)}{C(0)}.$$

This normalization is important because it ensures that $R(0) = 1$, providing a fixed reference point for perfect correlation. Moreover, it allows for a meaningful comparison between signals with different amplitudes or units. It isolates the temporal *structure* of the correlations from their overall magnitude. In physical terms, $R(\tau)$ tells us how predictable or 'memoryful' a signal is:

- If $R(\tau)$ decays rapidly to zero, the process is nearly random (has a short correlation time).
- If $R(\tau)$ oscillates or decays slowly, the process retains memory, indicating periodic or long-range correlations (e.g., vibrations, waves, or resonance phenomena).

Thus, the autocorrelation function is a fundamental diagnostic tool in time-series physics, signal processing, and experimental data analysis. It reveals hidden periodicities, quantifies temporal coherence, and connects directly to spectral power through the *Wiener-Khinchin theorem*.

4.3.2 Wiener–Khinchin theorem (statement and physical meaning)

The Wiener–Khinchin theorem relates the autocorrelation function of a stationary process to its Power Spectral Density (PSD). In continuous time,

$$S_x(f) = \int_{-\infty}^{\infty} R_x(\tau) e^{-2\pi i f \tau} d\tau,$$

i.e. the PSD is the Fourier transform of the autocorrelation function. In contrast,

$$R_x(\tau) = \int_{-\infty}^{\infty} S_x(f) e^{2\pi i f \tau} df.$$

The autocorrelation reveals time-domain periodicities and memory, while its Fourier transform (the PSD) shows how variance is distributed over frequency components. In experimental physics, the Power Spectral Density (PSD) provides an essential diagnostic of the processes generating a signal. Each feature in the PSD has a direct physical meaning:

- **Spectral peaks and resonances:** Sharp peaks in the PSD correspond to frequencies where the system oscillates or responds strongly. These peaks indicate *resonant frequencies* characteristic of harmonic oscillators, circuits, or mechanical systems. For example, in an RLC circuit, the resonant frequency appears as a narrow, tall peak in $S_x(f)$. Periodic signals, such as a rotating wheel or a modulated laser beam, also appear as distinct frequency lines in the PSD.
- **Noise color and spectral shape:** The general shape or “slope” of the PSD encodes the nature of the noise in the signal:
 - *White noise* has equal power at all frequencies, producing a flat PSD $S_x(f) = \text{const.}$ This means each time sample is uncorrelated with the others.
 - *Pink noise* ($1/f$ noise) decreases with frequency, implying stronger correlations at long timescales (slow drift or instability).
 - *Brown noise* ($1/f^2$) indicates an even smoother, slowly varying process such as random walks or integrated white noise.

Thus, the *color of the noise* seen in the PSD reveals the underlying dynamics or the dominant physical processes. By examining $S_x(f)$, an experimentalist can immediately recognize:

- Whether a system has resonances (peaks at specific frequencies). For example, in *laser interferometers* or *gravitational-wave detectors*, sharp PSD peaks correspond to mechanical resonances of mirrors or suspension systems.
- In *electronic circuits*, resonance peaks reveal filter characteristics or feedback oscillations.
- Whether unwanted vibrations, electrical pickup, or instrumental drift dominate.
- Whether a measured signal is consistent with random (white) noise or exhibits long-term correlations.

In summary, the PSD acts as a “frequency-domain fingerprint” of a signal. It simultaneously exposes periodicities (through peaks) and stochastic characteristics (through the spectral slope or noise color), enabling deep insight into the physical origin and quality of the measured data. For example in *solid-state and condensed-matter experiments*, noise color distinguishes between white shot noise, $1/f$ flicker noise, and thermal noise sources. White noise is the combination of thermal and shot noise, both of which are generally considered frequency-independent. Thermal noise (Johnson-Nyquist noise) is caused by the random thermal movement of electrons in a conductor and is proportional to temperature. Shot noise is caused by the discrete nature of charge carriers and is not temperature-dependent. In contrast, $1/f$ flicker noise is a low-frequency noise that increases as the frequency decreases, often originating from imperfections in semiconductors and other materials.

4.4 Matched Filtering

Matched filtering is a powerful technique for detecting a known signal hidden in noisy data. It is widely used across experimental physics and engineering, for example in radar and communications, spectroscopy, exoplanet transit searches, and gravitational-wave astronomy (for instance the LIGO and Virgo experiments). The key idea is to correlate the measured data with a template that matches the expected signal shape so the detector output is large only when the data contains the expected waveform and is properly time-aligned. In such experiment, the measured data stream $s(t)$ is modeled as

$$s(t) = A u(t - \tau_0) + n(t), \quad (4.1)$$

where $u(t)$ is a known template waveform (e.g., a theoretical model of the expected signal) which is normalized in energy unless otherwise stated, A is an unknown

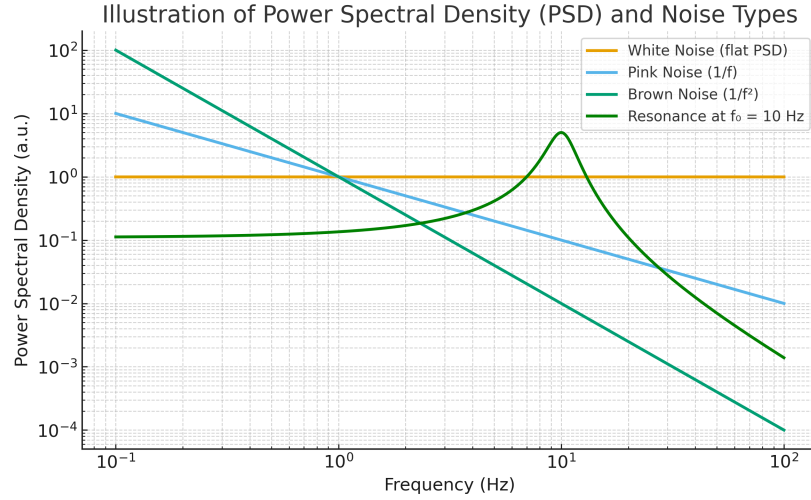


Figure 4.1: Illustration of Power Spectral Density (PSD) and noise types. White noise exhibits a flat spectrum, indicating uncorrelated fluctuations across all frequencies. Pink noise ($1/f$) and brown noise ($1/f^2$) display increasing correlation at low frequencies. The green curve shows a resonance at $f_0 = 10$ Hz, typical of an oscillatory or mechanical system responding strongly near its natural frequency.

amplitude scaling, τ_0 is the unknown time shift (arrival time) of the signal, and $n(t)$ is additive noise. In what follows we assume $n(t)$ is a zero-mean, stationary Gaussian process unless otherwise noted. Our objectives are (i) to decide whether the template "the signal" $u(t)$ is present in the noisy data $s(t)$, and (ii) to estimate its parameters such as A and τ_0 when it is present.

4.4.1 Building a Detection Statistic (White Noise Case)

To construct a linear detection statistic, apply a filter $q(t)$ to the data and form the scalar output

$$\Lambda = \int_{-\infty}^{\infty} s(t) q(t) dt. \quad (4.2)$$

Under the model (4.1), the expectation and variance of Λ are

$$\mathbb{E}[\Lambda] = \int A u(t - \tau_0) q(t) dt, \quad (4.3)$$

$$\text{Var}(\Lambda) = \mathbb{E}[(\Lambda - \mathbb{E}[\Lambda])^2] = \sigma_n^2 \int_{-\infty}^{\infty} q^2(t) dt, \quad (4.4)$$

where we have assumed white Gaussian noise with covariance $\langle n(t) n(t') \rangle = \sigma_n^2 \delta(t - t')$.

Define the signal-to-noise ratio (SNR) of the statistic Λ as

$$\text{SNR}[q] = \frac{\mathbb{E}[\Lambda \mid \text{signal present}]}{\sqrt{\text{Var}(\Lambda \mid \text{noise only})}} = \frac{A \int u(t - \tau_0) q(t) dt}{\sigma_n \sqrt{\int q^2(t) dt}}. \quad (4.5)$$

We design $q(t)$ to maximize the SNR. By the Cauchy–Schwarz inequality,

$$\left| \int u(t - \tau_0) q(t) dt \right| \leq \left(\int u^2(t - \tau_0) dt \right)^{1/2} \left(\int q^2(t) dt \right)^{1/2},$$

with equality if and only if $q(t)$ is proportional to $u(t - \tau_0)$. Therefore the SNR is maximized when

$$q(t) = K u(t - \tau_0), \quad (4.6)$$

for arbitrary scaling constant K . Choosing K to normalize the filter for convenience, we obtain the classical matched filter: the filter kernel is the template (time-shifted to the expected arrival time).

With the template energy normalized so that $\int u^2(t) dt = 1$, the maximum achievable SNR becomes

$$\text{SNR}_{\max} = \frac{A}{\sigma_n}. \quad (4.7)$$

The matched filter thus computes a correlation between the data and a time-shifted template. The statistic should respond strongly (large ρ or Λ) when $s(t)$ contains $A u(t - \tau_0)$ and weakly on average when the data contain only noise. In practice one evaluates the correlation as a function of the trial time shift τ ; the location of the peak provides an estimate of τ_0 and the peak height provides an estimate of the amplitude A in noise.

This setup captures a universal situation in physics: we have a known signal shape buried in random background noise, and we need to extract it as reliably as possible. Examples include:

- detecting gravitational waves in LIGO data,
- finding faint exoplanet transits in stellar light curves,
- measuring weak NMR or EPR signals in spectroscopy.

The filter $q(t)$ acts as a *matched detector*: it enhances the components of $s(t)$ that are consistent with $u(t - \tau_0)$ and suppresses those that are not. By “matching” the filter to the signal’s time-frequency structure, we ensure that the

SNR is maximized a principle known as matched filtering.

Example 4.1: White-noise matched filter (time-domain example)

As an example to illustrate the operation of a matched filter, consider a simple Gaussian pulse buried in white noise. Let the template signal be

$$u(t) = e^{-t^2/(2\sigma_u^2)},$$

centered at $t = 0$ and normalized so that $\int u^2(t) dt = 1$. The observed data are

$$s(t) = A u(t - \tau_0) + n(t),$$

where A is the amplitude, τ_0 is the true arrival time, and $n(t)$ is zero-mean Gaussian noise with variance σ_n^2 .

The matched filter statistic is computed for different time lags τ :

$$\rho(\tau) = \int s(t) u(t - \tau) dt,$$

which measures the similarity between the data and a shifted copy of the template. The maximum of $\rho(\tau)$ indicates where the signal best matches the template - hence the most likely arrival time. Hereunder a Python code to simulate this example.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 # Parameters
5 np.random.seed(0)
6 t = np.linspace(-5, 5, 1000)
7 sigma_u = 0.5
8 A, tau0, sigma_n = 2.0, 1.0, 0.5
9
10 # Template and data
11 u = np.exp(-t**2 / (2*sigma_u**2))
12 u /= np.sqrt(np.sum(u**2)) # normalize
13 n = np.random.normal(0, sigma_n, size=t.size)
14 s = A*np.exp(-(t - tau0)**2 / (2*sigma_u**2)) + n
15
16 # Matched filter correlation
17 rho = np.correlate(s, u, mode="same")
18
19 # Plot
20 plt.figure(figsize=(8,5))

```

```

21 plt.plot(t, s, label="Observed signal s(t)", color="gray", alpha
    =0.6)
22 plt.plot(t, A*np.exp(-(t - tau0)**2 / (2*sigma_u**2)), 'k--',
    label="True signal")
23 plt.plot(t, rho/np.max(rho)*A, 'r', label="Matched filter output (
    scaled)")
24 plt.axvline(tau0, color="red", linestyle="--", alpha=0.7)
25 plt.legend()
26 plt.xlabel("Time t")
27 plt.ylabel("Amplitude")
28 plt.title("Matched Filter Response to a Gaussian Signal in Noise")
29 plt.show()

```

Code 4.3: Matched filter detection of a Gaussian pulse

This code produces:

- a normalized Gaussian pulse $u(t)$ which serves as the template.
- the observed data $s(t)$ that contains the same pulse delayed by τ_0 and added noise.
- the correlation $\rho(\tau)$ is computed to measure how similar $s(t)$ is to a shifted version of $u(t)$.
- the output $\rho(\tau)$ peaks sharply near τ_0 , indicating detection of the signal at that time.

This example illustrates how matched filtering computes how strongly the data resemble a known pattern as a function of delay. In gravitational wave physics, for instance, thousands of theoretical waveforms (templates) are correlated with detector data to locate weak signals hidden in noise. The peak of $\rho(\tau)$ corresponds to the best alignment, hence the moment the signal truly arrived.

4.4.2 Colored Noise and Frequency-Domain matched filter

Real experimental noise is seldom white. For stationary colored noise described by a one-sided power spectral density (PSD) $S_n(f)$, the optimal filter must down-weight noisy frequency bands by weighting them with the inverse of the noise PSD. Define the noise-weighted inner product of two real functions $a(t)$ and $b(t)$ as

$$(a|b) \equiv 4 \operatorname{Re} \int_0^\infty \frac{\tilde{a}^*(f)\tilde{b}(f)}{S_n(f)} df,$$

where $\tilde{a}(f)$ is the Fourier transform of $a(t)$ and * denotes complex conjugation. For these definitions, the matched-filter SNR for a template h and data s is

$$\rho = \frac{(s|h)}{\sqrt{(h|h)}}.$$

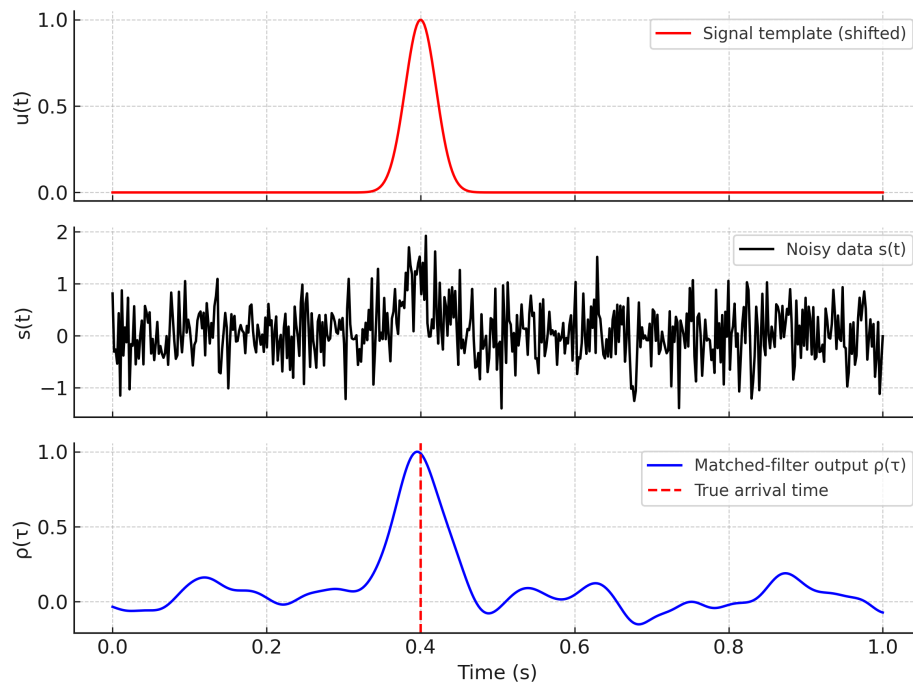


Figure 4.2: Matched filter detection of a Gaussian pulse buried in white noise. The red dashed line marks the true signal arrival time τ_0 . The matched filter output (red curve) peaks at the correct location, indicating successful signal recovery.

By replacing time-domain integrals by frequency-domain inner products and carrying through an analogous Cauchy–Schwarz argument (now with S_n^{-1} weighting) shows that the optimal filter is proportional to $\tilde{h}(f)/S_n(f)$ (i.e., whitened template). This is the basis for real gravitational-wave searches, where templates are weighted by the detector noise curve.

4.4.3 Physical importance

Matched filtering is the *optimal* linear method for detecting a known waveform in stationary Gaussian noise — it maximizes SNR. Its optimality and ease of implementation are why it is widely used in experimental physics (LIGO, pulsar searches, radar, spectroscopy). However, it requires an accurate template; model mismatch reduces detection efficiency.

Example 4.2: Matched filtering in practice

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from scipy.signal import correlate
4
5 np.random.seed(3)
6 t = np.linspace(0, 5, 1000)
7
8 # Template: localized sine-Gaussian pulse centered near t=2.5 s
9
10 h = np.sin(2*np.pi*10*t) * np.exp(-((t-2.5)**2)/0.1)
11 noise = 0.5 * np.random.randn(len(t))
12 data = noise.copy()
13
14 # Inject a weaker scaled version of the template between indices
15   400 and 600
16
17 data[400:600] += 0.6 * h[400:600]
18
19 mf = correlate(data, h, mode='same')
20 mf /= np.max(np.abs(mf)) # normalize for plotting
21
22 plt.figure(figsize=(8,4))
23 plt.plot(t, data, label="Noisy data")
24 plt.plot(t, mf, "r-", label="Matched-filter output (normalized)")
25 plt.xlabel("Time [s]"); plt.ylabel("Amplitude / Correlation")
26 plt.legend(); plt.title("Matched filtering detects injected
   template")
27 plt.show()

```

Code 4.4: Matched filtering example (time-domain correlate)

Practical considerations and diagnostics

4.4.4 False-alarm rate and detection threshold

When using a detection statistic Λ (e.g., matched-filter SNR), set a threshold Λ_{th} to limit false alarms. The false-alarm probability P_{FA} is

$$P_{\text{FA}} = \int_{\Lambda_{\text{th}}}^{\infty} p(\Lambda|H_0), d\Lambda,$$

where H_0 denotes the noise-only hypothesis. For Gaussian noise and a linear statistic with known variance, P_{FA} can be computed analytically; for real detectors, one often estimates it with background trials (time slides, off-source stretches).

4.4.5 Template bank and trials factor

When searching over many templates (e.g., many parameter combinations), the trials factor increases the effective false-alarm rate: thresholds must be adjusted (or p-values corrected) to maintain a global false-alarm probability.

Chapter summary and importance

- Confidence intervals quantify parameter uncertainty and depend on sample size and noise. Use t -statistics when noise variance is estimated from data.
- The design matrix in regression controls parameter covariance; experiment design (spread of x values) matters greatly.
- Correlation measures (Pearson r) quantify linear association; the Fisher z -transform gives approximately Gaussian errors on r for CI construction.
- Autocorrelation and the PSD are Fourier pairs; spectral methods reveal periodicities and noise color.
- Matched filtering is the optimal linear detector for known templates in Gaussian noise (white or colored after appropriate whitening). Its broad use in physics is due to provable optimality and computational feasibility.

Problems (focus: confidence and uncertainty)

All solutions are available in Appendix ??.

Problem 1. Mean CI by simulation

Simulate $n = 10, 20, 50, 100$ draws from a Gaussian with mean $\mu = 0$ and $\sigma = 1$. For each n :

- Compute the 95% CI using the t -distribution formula.
- Empirically estimate coverage by repeating the experiment 5000 times and reporting the fraction of CIs that contain μ .
- Discuss convergence of empirical coverage to nominal coverage.

Problem 2. Regression CI and experiment design

Using the linear regression setup derive analytically how the variance of the slope $\text{Var}(\hat{m})$ changes with:

$$x_i \in [a, b] \quad (\text{uniformly sampled}).$$

Show that spreading x_i over a wider interval reduces $\text{Var}(\hat{m})$, and illustrate numerically.

Problem 3. Correlation confidence

Generate two variables with true correlation $\rho = 0.5$, $N = 30$ samples:

- Compute the sample r and its 95% CI using the Fisher transform.
- Compare with bootstrap CI (resampling with replacement).
- Discuss differences and when each method is preferable.

Problem 4. Propagating uncertainty for correlated variables

Suppose you measure two quantities x and y with uncertainties and covariance:

$$\begin{pmatrix} \sigma_x^2 & \text{Cov}(x, y) \\ \text{Cov}(x, y) & \sigma_y^2 \end{pmatrix}.$$

For the derived quantity $z = x, y$, derive the propagated variance σ_z^2 via linear error propagation (first-order Taylor expansion), and discuss the role of the covariance term.

Problem 5. Bootstrap confidence for correlation (coding)

Using a sample of length $N = 50$ with unknown dependence, use bootstrap resampling to estimate a 95% CI for Pearson r and compare to Fisher CI. Comment on coverage and skewness of the bootstrap distribution.

Problem 6. Matched-filter detection threshold

Simulate many noise-only realizations of the matched-filter statistic for a fixed template and estimate the distribution of the maximum SNR in each trial. From these simulations, estimate the SNR threshold that yields a false-alarm rate of 10^{-3} . Discuss dependence on data length and template bandwidth. analysis.

Problem 7. Autocorrelation and the Wiener–Khinchin theorem

Generate a synthetic time series composed of a sinusoidal signal plus additive white noise:

$$x(t_i) = \sin(2\pi f_0 t_i) + n_i, \quad n_i \sim \mathcal{N}(0, \sigma^2),$$

for $f_0 = 5$ Hz, $\sigma = 0.5$, and $N = 1000$ samples at $\Delta t = 0.01$ s.

- Compute and plot the empirical autocorrelation function $R(\tau)$ of $x(t)$.
- Compute the power spectral density (PSD) using the FFT, and verify numerically that the PSD is approximately the Fourier transform of $R(\tau)$ (Wiener–Khinchin theorem).
- Identify the location of spectral peaks and interpret them in terms of periodicities seen in $R(\tau)$.
- Discuss how the presence of white noise affects both the autocorrelation and the PSD.

Hint: Normalize both $R(\tau)$ and $S_x(f)$ to unit variance for easier comparison.

Problem 8. Noise color and correlation time

Simulate three types of stochastic signals of equal length $N = 5000$:

- White noise ($S(f) \propto f^0$),
- Pink noise ($S(f) \propto 1/f$),

- Brown noise ($S(f) \propto 1/f^2$).
- Estimate and plot the autocorrelation function $R(\tau)$ for each case.
- Compute and plot their PSDs on log-log axes and measure the spectral slopes.
- Estimate a correlation time τ_c from the lag where $R(\tau)$ drops to $1/e$.
- Discuss how noise “color” relates to the signal’s memory and temporal structure.

Optional coding extension: generate colored noise via frequency-domain filtering of white noise, i.e. multiply Fourier amplitudes by $1/f^\alpha$ before inverse transforming.

Problem 9. Detection efficiency and ROC analysis (extension of matched filtering)

Extend the simulation of matched-filter outputs from Problem 4.4.5 to include true signal injections.

- For each trial, generate a noisy data segment containing either:
 1. noise only (H_0 case), or
 2. noise plus a known template signal (H_1 case) with amplitude A and known arrival time.
- Compute the matched-filter statistic ρ for each realization and record whether ρ exceeds a chosen detection threshold ρ_{th} .
- By varying ρ_{th} , estimate empirically:
 - the false-alarm probability $P_{\text{FA}} = P(\rho > \rho_{\text{th}} | H_0)$,
 - the detection probability $P_{\text{D}} = P(\rho > \rho_{\text{th}} | H_1)$.
- Plot P_{D} versus P_{FA} to obtain a Receiver Operating Characteristic (ROC) curve.
- Investigate how P_{D} changes with signal amplitude A , data length, and noise level.

Discussion:

- Interpret the ROC curve as quantifying the tradeoff between sensitivity and reliability in detection experiments.
- Compare the empirically derived ROC to the theoretical expectation for Gaussian noise (analytical P_{FA} and P_{D} for a given SNR).
- Comment on how matched-filter optimality ensures the ROC curve is maximized for linear detectors in Gaussian noise.

Optional coding extension: use Python to generate 10^4 trials and compute ROC curves for multiple signal amplitudes. Overlay curves for different A values and label the corresponding SNRs.

Chapter 5

Gravitational-Wave Data Analysis (LIGO Open Data)

5.1 Introduction

Gravitational waves are ripples in spacetime predicted by Einstein’s General Theory of Relativity. They are produced by cataclysmic astrophysical events such as the merger of black holes or neutron stars. When such waves pass through Earth, they cause tiny distortions in spacetime that can be detected by laser interferometers such as the Laser Interferometer Gravitational-Wave Observatory (LIGO).

In this chapter, we will learn how to import, process, and visualize gravitational wave data using Python. We will make heavy use of the **NumPy**, **Matplotlib**, and **SciPy** libraries, following examples from real open data released by the LIGO Scientific Collaboration.

5.2 Importing Data

LIGO open data can be accessed from public repositories. In practice, we work with prepackaged datasets for learning purposes. The following snippet shows how we would import libraries for handling numerical data and visualization:

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from scipy import signal
```

Code 5.1: Importing libraries for gravitational wave analysis

These libraries will support numerical operations (**NumPy**), plotting (**Matplotlib**), and signal processing (**SciPy**).

5.3 Gravitational Wave Signals

A passing gravitational wave induces changes in the relative lengths of the interferometer arms. These changes are recorded as a time-series strain signal, typically denoted $h(t)$. The signals are extremely small ($\Delta L/L \sim 10^{-21}$), so sophisticated statistical and signal-processing techniques are necessary to identify them against noise.

5.4 Visualizing Sample Data

Let us suppose we have a simple gravitational wave signal stored in a file. The first step is to load it into Python and plot it:

```

1 # Example data (simulated for teaching)
2 t = np.linspace(0, 1, 2000) # 1 second of data at 2 kHz
3 h = np.sin(80 * np.pi * t) * np.exp(-t*4) # decaying sinusoid as
   mock GW signal
4
5 plt.plot(t, h)
6 plt.xlabel("Time [s]")
7 plt.ylabel("Strain h(t)")
8 plt.title("Simulated Gravitational Wave Signal")
9 plt.show()

```

Code 5.2: Loading and plotting a sample gravitational wave strain signal

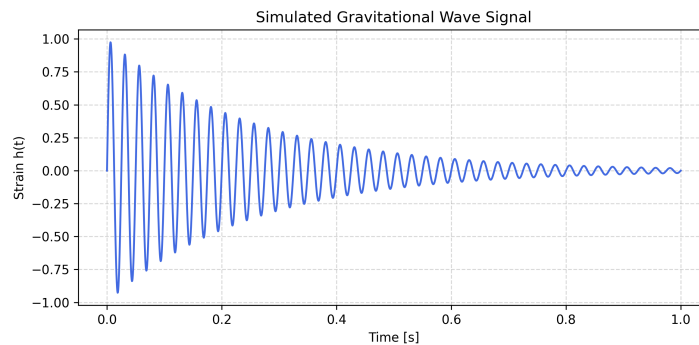


Figure 5.1: Output: Simulated gravitational wave signal, showing a decaying sinusoidal “chirp” form.

This simple example mimics the chirp signal seen in binary mergers.

5.5 Frequency Analysis

It is often useful to analyze the data in the frequency domain. The Fast Fourier Transform (FFT) lets us decompose $h(t)$ into its frequency components.

```

1 from numpy.fft import rfft, rfftfreq
2
3 N = len(t)
4 dt = t[1] - t[0]
5
6 freqs = rfftfreq(N, dt)
7 spectrum = np.abs(rfft(h))
8
9 plt.plot(freqs, spectrum)
10 plt.xlabel("Frequency [Hz]")
11 plt.ylabel("Amplitude")
12 plt.title("Frequency Spectrum of Gravitational Wave Signal")
13 plt.show()

```

Code 5.3: Fourier transform of a gravitational wave signal

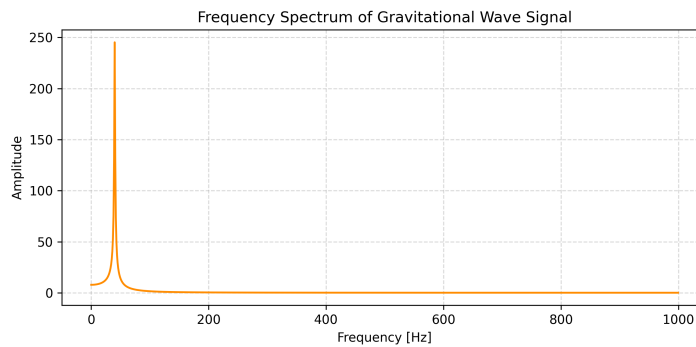


Figure 5.2: Output: Frequency spectrum of the simulated gravitational wave signal.

This method reveals the characteristic frequency content of the signal.

5.6 Filtering and Noise Reduction

Real LIGO data is noisy. Filtering is necessary to extract signals from noise. A simple band-pass filter can isolate the frequencies where gravitational waves are expected.

```

1 from scipy.signal import butter, filtfilt
2
3 def bandpass(data, fs, low, high):

```

```

4     nyq = 0.5 * fs
5     b, a = butter(4, [low/nyq, high/nyq], btype='band')
6     return filtfilt(b, a, data)
7
8 fs = 2000 # sampling rate
9 filtered_h = bandpass(h, fs, 30, 300)
10
11 plt.plot(t, filtered_h)
12 plt.xlabel("Time [s]")
13 plt.ylabel("Strain h(t)")
14 plt.title("Filtered Gravitational Wave Signal")
15 plt.show()

```

Code 5.4: Applying a band-pass filter

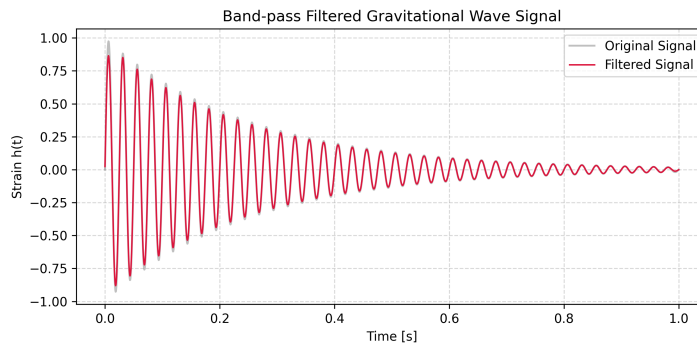


Figure 5.3: Output: Band-pass filtered gravitational wave signal.

5.7 Chapter Summary

In this chapter, we introduced the concept of gravitational waves and how they are detected. We learned how to import and visualize strain signals, compute their frequency spectrum, and apply filtering techniques to remove noise. These are the foundations for more advanced analyses, such as matched filtering, parameter estimation, and statistical hypothesis testing in gravitational wave astronomy.

Problems

Problem 1. Simulated Chirp Signal

Create a simulated chirp signal with frequency increasing from 30 Hz to 200 Hz over 1 second.

- Plot the signal in the time domain.
- Compute and plot its frequency spectrum.

Solution available in Appendix ??, see Problem ??.

Problem 2. Noise Filtering

Generate a sine wave at 50 Hz and add Gaussian noise.

- Apply a band-pass filter to recover the sine wave.
- Compare the original clean signal with the noisy and filtered versions.

Solution available in Appendix ??, see Problem ??.

Problem 3. Real Data Exploration (Challenge)

Download a real open dataset from LIGO Open Science Center (GW150914).

- Plot the raw strain signal.
- Apply filtering to enhance the gravitational wave signal.
- Compare with published LIGO plots of the same event.

Solution available in Appendix ??, see Problem ??.

Chapter 6

Jet Physics, Bayesian Statistics, and Likelihood

High-energy physics experiments, such as those at the Large Hadron Collider (LHC), produce complex final states containing thousands of particles. Among these, *jets*, collimated sprays of hadrons arising from quarks and gluons, are central objects of study. This chapter introduces the physics of jets, their experimental reconstruction, and the statistical tools required for modern collider analysis.

6.1 Jets in High-Energy Physics

Jets bridge theory and experiment: the hard-scattered partons predicted by QCD are not directly observable, yet they manifest experimentally as sprays of hadrons measured in detectors. Understanding how jets form, how they are reconstructed, and how they are characterized is fundamental for precision measurements and new-physics searches.

6.1.1 From Partons to Jets: Showering, Hadronization, and Decay

A jet originates from a high-energy quark or gluon produced in a hard interaction. Its evolution from a single energetic parton to hundreds of hadrons proceeds through three major stages.

1. Parton Showering (Perturbative QCD)

Immediately after the hard scattering, an outgoing quark or gluon radiates additional partons. These emissions follow the QCD splitting processes:

$$q \rightarrow qg, \quad g \rightarrow gg, \quad g \rightarrow q\bar{q}.$$

The emission pattern is dominated by *soft* and *collinear* radiation, and is governed by the strong coupling $\alpha_s(Q)$. Parton showering is perturbative and is modeled using DGLAP evolution. Monte Carlo tools such as **PYTHIA**, **HERWIG**, and **Sherpa** implement realistic shower cascades.

2. Hadronization (Non-perturbative QCD)

As the partons' virtuality decreases to the scale $Q \sim \Lambda_{\text{QCD}} \sim 1 \text{ GeV}$, perturbation theory fails and partons must transition to color-neutral hadrons. Two major models describe this:

String fragmentation (PYTHIA). A color flux tube forms between separating partons and breaks into hadrons.

Cluster hadronization (HERWIG). Color-singlet clusters form and decay into hadrons.

This process produces a jet of pions, kaons, baryons, and other hadrons aligned with the original parton direction.

3. Hadron Decays

Many hadrons decay before reaching the detector. For example,

$$\pi^0 \rightarrow \gamma\gamma,$$

leading to large electromagnetic deposits. Decay chains enrich the jet with a mixture of charged and neutral particles.

Jet Structure Illustration

Figure 6.1 depicts a typical jet: hard-core particles near the center, surrounded by softer radiation from the late parton shower, plus diffuse contributions from underlying event and pileup.

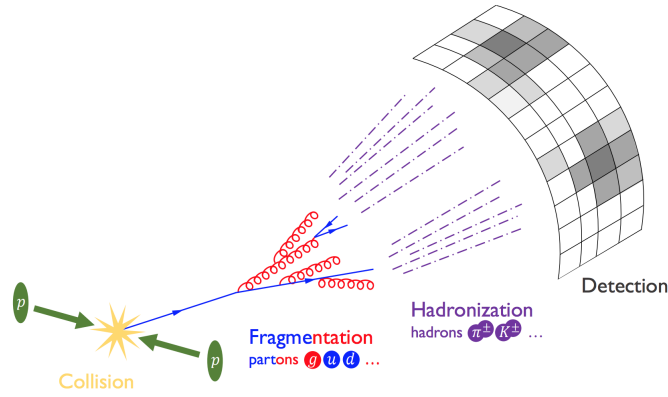


Figure 6.1: Schematic illustration of a jet: core from leading partons, intermediate region from showering, and wide-angle soft radiation. (To be replaced by generated figure.)

6.1.2 From Detector Signals to Jets: Clustering Algorithms

Jets are not directly measured. The detector records:

- charged particle tracks in the inner tracker,
- electromagnetic clusters in the ECAL,
- hadronic energy deposits in the HCAL.

Reconstructing jets requires combining these measurements into physics objects, and then clustering them into jets.

1. Calorimeter and Particle-Flow Clustering

Detectors perform initial clustering of raw calorimeter cells:

- ATLAS uses *topological clusters* (topoclusters),
- CMS uses *particle-flow* (PF) candidates that combine tracker + calorimeter information.

This step reduces millions of detector cells into a few thousand reconstructed objects representing stable particles.

2. The Anti- k_T Jet Algorithm

The standard jet-definition at the LHC is the anti- k_T algorithm. Given reconstructed objects (PF candidates or clusters), define distances

$$d_{ij} = \min(p_{T,i}^{-2}, p_{T,j}^{-2}) \frac{\Delta R_{ij}^2}{R^2}, \quad d_{iB} = p_{T,i}^{-2},$$

where $\Delta R_{ij} = \sqrt{(\Delta\eta)^2 + (\Delta\phi)^2}$.

Algorithm.

1. Compute all d_{ij} and d_{iB} .
2. Find the smallest distance.
3. If it is a d_{ij} , merge objects i and j .
4. If it is a d_{iB} , declare object i a final jet.
5. Repeat.

Why anti- k_T is widely used.

- Produces nearly circular jets in (η, ϕ) .
- Stable against additional soft or collinear particles.
- Infrared and collinear safe (theoretically robust).
- Resistant to pileup.

6.1.3 Jet Observables: p_T , Rapidity, Pseudorapidity, and Jet Mass

After clustering, each jet is assigned kinematic quantities.

Transverse Momentum p_T

The beam axis defines the z -direction. The transverse momentum is

$$p_T = p \sin \theta,$$

where θ is the polar angle relative to the beam.

Importance.

- In proton–proton collisions, initial-state p_z is unknown.
- p_T is directly measured and conserved in the transverse plane.
- Trigger and analysis cuts always rely on p_T .

Rapidity

Defined as

$$y = \frac{1}{2} \ln \left(\frac{E + p_z}{E - p_z} \right).$$

Why rapidity matters.

- Differences in rapidity are Lorentz invariant under boosts along the beam.
- Many theoretical predictions (PDFs, cross sections) use rapidity.
- Describes longitudinal motion of particles in a natural way.

Pseudorapidity

For ultrarelativistic particles, $E \approx p$, and the angle-based variable

$$\eta = -\ln \left(\tan \frac{\theta}{2} \right)$$

is used.

Why experiments use η instead of y .

- Detectors measure angles more accurately than energies.
- For high-energy particles, $y \approx \eta$.
- Detector acceptance and segmentation are defined in η .

Jet Mass

The four-momentum of a jet is the sum of the four-momenta of its constituents:

$$p_{\text{jet}}^\mu = \sum_{i \in \text{jet}} p_i^\mu.$$

Then

$$m_{\text{jet}}^2 = E^2 - |\vec{p}|^2.$$

Importance.

- Identifying boosted W , Z , or Higgs bosons that decay hadronically.
- Discriminating QCD jets from multi-prong decay jets (jet substructure).

6.2 Likelihood and Statistical Inference in HEP

Jet measurements and searches for new physics rely heavily on the *likelihood function*, which encodes all statistical information contained in the data. In high-energy physics, likelihood-based inference is the standard for parameter estimation, confidence intervals, and hypothesis testing.

6.2.1 The Likelihood Function

Suppose a measurement yields data D (e.g. binned jet p_T spectrum, jet mass distribution, or event counts). Let θ denote the parameters of interest (cross sections, efficiencies, jet energy scale, nuisance parameters, etc.). The likelihood is

$$\mathcal{L}(\theta \mid D) = P(D \mid \theta),$$

the probability of obtaining the observed data given parameters θ . For binned event counts, where each bin i is Poisson-distributed,

$$\mathcal{L}(\theta) = \prod_{i=1}^N \frac{\nu_i(\theta)^{n_i} e^{-\nu_i(\theta)}}{n_i!},$$

where n_i is the observed number of events and $\nu_i(\theta)$ is the expected yield in bin i .

The log-likelihood,

$$\ln \mathcal{L}(\theta) = \sum_{i=1}^N [n_i \ln \nu_i(\theta) - \nu_i(\theta) - \ln(n_i!)],$$

is convenient for numerical optimization.

6.2.2 Maximum Likelihood Estimation (MLE)

The *maximum-likelihood estimator* (MLE) is

$$\hat{\theta} = \arg \max_{\theta} \mathcal{L}(\theta).$$

Under mild regularity conditions, $\hat{\theta}$ is asymptotically unbiased and normally distributed with variance

$$\text{Var}(\hat{\theta}) = \left[-\frac{\partial^2 \ln \mathcal{L}}{\partial \theta^2} \Big|_{\theta=\hat{\theta}} \right]^{-1}.$$

In multi-parameter problems, the covariance matrix is the inverse of the Fisher information matrix.

6.2.3 Likelihood-Ratio Tests

To compare hypotheses H_0 and H_1 we use the test statistic

$$\lambda = -2 \ln \left(\frac{\mathcal{L}(\theta_0)}{\mathcal{L}(\hat{\theta})} \right),$$

which, by Wilks' theorem, is asymptotically χ^2 -distributed with degrees of freedom equal to the number of constrained parameters.

Applications include:

- significance of a resonance peak in the jet mass spectrum,
- testing for boosted boson signatures,
- jet energy scale calibration parameters,
- comparing QCD modeling scenarios.

6.2.4 Likelihoods With Systematic Uncertainties

Systematics (JES/JER, luminosity, detector efficiencies, theoretical scale uncertainties) are encoded via *nuisance parameters* ν . The extended likelihood is

$$\mathcal{L}(\theta, \nu) = \mathcal{L}_{\text{data}}(\theta, \nu) \prod_j \pi_j(\nu_j),$$

where π_j are constraint terms (Gaussians, log-normals, etc.).

Profiling eliminates nuisance parameters:

$$\mathcal{L}_p(\theta) = \mathcal{L}(\theta, \hat{\nu}(\theta)).$$

Profiling is the basis of the LHC standard approach for cross section measurements and new-physics searches.

6.3 Bayesian Statistics in Jet Physics

Bayesian inference provides a complementary—and often more intuitive—approach for parameter estimation, uncertainty quantification, and unfolding.

6.3.1 Bayes' Theorem

Given prior $p(\theta)$ and likelihood $\mathcal{L}(\theta)$, the posterior is

$$p(\theta \mid D) = \frac{\mathcal{L}(\theta) p(\theta)}{\int \mathcal{L}(\theta') p(\theta') d\theta'}.$$

The denominator is the *evidence*.

6.3.2 Posterior Summaries

- Posterior mean: $\mathbb{E}[\theta \mid D]$
- Posterior mode: MAP estimate
- Credible intervals: central or highest-posterior-density (HPD)

These intervals differ conceptually from frequentist confidence intervals: Bayesian intervals give the probability that θ lies within the interval.

6.3.3 Choosing Priors

Common priors in jet-physics analyses:

- Flat prior in cross section σ
- Log-normal prior for efficiencies or luminosity
- Gaussian priors for calibration uncertainties (JES/JER)

Priors should be chosen to reflect physical constraints, theoretical knowledge, or detector limitations.

6.3.4 Markov Chain Monte Carlo (MCMC)

Posterior integrals are evaluated using MCMC methods:

- Metropolis–Hastings
- Hamiltonian Monte Carlo (HMC)
- No-U-Turn Sampler (NUTS), used in PyMC3 / Stan

MCMC is used for:

- jet energy calibration with nuisance parameters,
- unfolding jet spectra,
- PDF fits and cross-section extraction,
- global SMEFT fits.

6.4 Applications to Jet Measurements

6.4.1 Jet Energy Scale (JES) Calibration

The JES parameter κ relates reconstructed and true energies:

$$E_{\text{reco}} = \kappa E_{\text{true}}.$$

A likelihood fit can extract κ using $\gamma + \text{jet}$ or $Z + \text{jet}$ balancing.

6.4.2 Jet Mass Fits

Boosted hadronic decays (W/Z/Higgs/top) appear as peaks in jet mass. One fits signal+background templates:

$$\mathcal{L}(\theta_s, \theta_b) = \prod_i \text{Poisson}(n_i \mid \theta_s S_i + \theta_b B_i).$$

Extracting θ_s yields the boson production rate; nuisance parameters capture shape and scale systematics.

6.4.3 Searches for New Physics

Examples:

- dijet resonance searches,
- contact interaction limits,
- monojet searches for dark matter.

All rely on likelihood-ratio test statistics for exclusion or discovery.

6.5 Practical Examples (Python)

6.5.1 Example: Simple Poisson Likelihood Fit

```

1 import numpy as np
2 from scipy.optimize import minimize
3
4 # Toy jet count data
5 obs = np.array([12, 18, 25, 15])
6 x = np.array([200, 300, 400, 500]) # jet pT bins
7
8 def expected(mu):
9     return mu * np.exp(-x/500) # model
10
11 def neglogL(mu):
12     lam = expected(mu)
13     return -np.sum(obs*np.log(lam) - lam)
14
15 result = minimize(neglogL, x0=[20])
16 print("MLE:", result.x)

```

6.5.2 Example: Bayesian Posterior for Jet Rate

```

1 import numpy as np
2 from scipy.stats import gamma
3
4 obs = 40
5 prior_alpha = 1
6 prior_beta = 1
7
8 posterior = gamma(a=prior_alpha+obs, scale=1/(prior_beta+1))
9 print("Posterior mean =", posterior.mean())

```

6.6 Summary

This chapter introduced:

- The physics of jet formation and reconstruction,
- Jet clustering algorithms with emphasis on anti- k_T ,
- Jet observables (p_T , rapidity, η , jet mass),
- Likelihood-based statistical inference,
- Bayesian methods and MCMC,
- Applications to JES calibration, jet mass fits, and searches.

Jets are essential tools in collider physics, and likelihood-based analyses are the foundation for extracting physical parameters and testing the Standard Model at modern collider experiments.

Problems

Problem 1: Poisson Likelihood for Jet Counts

A jet analysis observes the following event counts in four transverse-momentum bins:

$$n = (12, 18, 25, 15).$$

The bin centers are

$$p_T = (200, 300, 400, 500) \text{ GeV}.$$

Assume the expected number of events in each bin follows the model

$$\nu_i(\mu) = \mu e^{-p_{T,i}/500},$$

where μ is an unknown normalization parameter.

1. Write down the Poisson likelihood $\mathcal{L}(\mu)$ for the observed data.
2. Derive the log-likelihood $\ln \mathcal{L}(\mu)$.
3. Find the maximum-likelihood estimate $\hat{\mu}$ numerically.
4. Sketch or plot the likelihood (or log-likelihood) as a function of μ .

Hints.

- Use the Poisson likelihood for binned data introduced in Section 6.2.
- The factorial terms $\ln(n_i!)$ do not depend on μ and can be dropped when maximizing the likelihood.
- The MLE satisfies $\partial \ln \mathcal{L} / \partial \mu = 0$ or can be found using numerical minimization of $-\ln \mathcal{L}$.

Problem 2: Likelihood-Ratio Test and Significance

Using the model and data from Problem 1, consider the hypotheses

$$H_0 : \mu = 0 \quad (\text{background only}), \quad H_1 : \mu \geq 0 \quad (\text{signal} + \text{background}).$$

1. Define the likelihood-ratio test statistic

$$q_0 = -2 \ln \frac{\mathcal{L}(\mu = 0)}{\mathcal{L}(\hat{\mu})}.$$

2. Compute q_0 using the MLE $\hat{\mu}$.
3. Assuming Wilks' theorem applies, estimate the significance Z in units of standard deviations.
4. Discuss when the asymptotic χ^2 approximation may not be valid.

Hints.

- For a single parameter, q_0 approximately follows a χ^2 distribution with one degree of freedom.
- The significance can be estimated using $Z \approx \sqrt{q_0}$.
- The asymptotic approximation may fail for small data samples or parameters near physical boundaries (e.g. $\mu \geq 0$).

Problem 3: Systematic Uncertainties and Profiling

Assume the expected yield depends on a jet energy scale (JES) factor κ :

$$\nu_i(\mu, \kappa) = \mu \kappa e^{-p_{T,i}/500}.$$

The JES uncertainty is modeled by a Gaussian constraint

$$\pi(\kappa) = \mathcal{N}(\kappa \mid 1, 0.05).$$

1. Write down the full likelihood $\mathcal{L}(\mu, \kappa)$.
2. Explain how κ acts as a nuisance parameter.
3. Define the profile likelihood $\mathcal{L}_p(\mu)$.
4. Explain qualitatively how including κ affects the uncertainty on μ .

Hints.

- Follow the construction of likelihoods with nuisance parameters in Section 6.4.
- The full likelihood is the product of the Poisson likelihood and the constraint term.
- Profiling means maximizing the likelihood with respect to κ for fixed μ .
- Systematic uncertainties typically broaden likelihoods and reduce significance.

Problem 4: Bayesian Inference for a Jet Rate

Assume a single-bin Poisson measurement with observed count $n = 40$. Let μ denote the expected mean event count.

1. Using a flat prior $p(\mu) \propto 1$ for $\mu \geq 0$, write down the posterior distribution $p(\mu | n)$.
2. Identify the functional form of the posterior.
3. Compute the posterior mean and variance.
4. Determine the central 68% Bayesian credible interval.

Hints.

- Use Bayes' theorem from Section 6.3.
- A Poisson likelihood with a flat prior leads to a Gamma posterior.
- Standard formulas for the mean and variance of a Gamma distribution may be used.
- A 68% credible interval contains 68% of the posterior probability.

Problem 5: Markov Chain Monte Carlo (MCMC)

In realistic jet analyses, the posterior may depend on many parameters.

1. Explain why numerical integration of posteriors becomes impractical in high-dimensional parameter spaces.
2. Describe the Metropolis–Hastings algorithm in words.
3. Explain how one assesses convergence and autocorrelation of MCMC chains.
4. State two advantages of Hamiltonian Monte Carlo (HMC) or NUTS over random-walk Metropolis.

Hints.

- The dimensionality of the parameter space grows with the number of nuisance parameters.
- MCMC generates samples whose density approximates the posterior distribution.
- Diagnostics include trace plots, autocorrelation functions, and multiple chains.
- HMC uses gradient information to explore parameter space more efficiently.

Looking Ahead

Chapter 7

Hypothesis Testing in Particle Physics (Higgs Search I)

High-energy physics experiments carry out hypothesis testing on a scale and at a level of statistical rigor unmatched in any other scientific field. The 2012 discovery of the Higgs boson—achieved independently by the ATLAS and CMS collaborations—is a canonical example of a high-stakes hypothesis test.

In this chapter, we introduce the logic of hypothesis testing as applied in particle physics, using the Higgs boson searches in the

$$H \rightarrow \gamma\gamma \quad \text{and} \quad H \rightarrow ZZ^* \rightarrow 4\ell$$

channels as concrete case studies. Chapter 8 will continue with confidence levels (CLs), exclusion limits, the Look-Elsewhere Effect (LEE), and combined channels.

7.1 The General Structure of a Particle Physics Search

A standard LHC search follows a well-defined statistical workflow. Although different analyses (Higgs, SUSY, exotica, EFT) may differ in their physics motivation, *the statistical structure is universal*.

1. Define hypotheses.

The goal of a search is to determine whether the observed data are compatible with the Standard Model background alone, or whether they contain evidence for a new signal. We therefore define:

$$H_0 : \text{background only}, \quad H_1 : \text{signal} + \text{background}.$$

In the Higgs searches discussed later, H_0 corresponds to the continuum Standard Model backgrounds (diphoton background, $ZZ^{(*)}$ background), while H_1 includes a Higgs resonance at mass m_H .

2. Build discriminating observables.

A discriminating observable separates signal from background. In resonance searches, the most powerful observable is the reconstructed invariant mass of the final state:

- $m_{\gamma\gamma}$ for $H \rightarrow \gamma\gamma$,
- $m_{4\ell}$ for $H \rightarrow ZZ^* \rightarrow 4\ell$.

It make sense beacause the Higgs produces a narrow peak at m_H , whereas the backgrounds yield smooth distributions.

Also in other(or even this) analyses, we may use:

- jet observables (p_T , mass, rapidity)
- missing transverse energy E_T^{miss} ,
- multivariate classifiers (BDTs, neural nets).

3. Model signal and background PDFs.

To perform statistical inference, we need probability density functions (PDFs) for the distributions of the discriminating observable under both hypotheses:

- **Signal PDF:** usually a narrow resonance model (Gaussian or Crystal Ball); see Section 6.3.4 for examples of PDF choice.
- **Background PDF:** smooth analytic function (exponential, polynomial, power-law), or nonparametric distribution from simulation, depending on the channel.

Associated parameters such as shape, normalization, and calibration uncertainties later become *nuisance parameters* in the likelihood (see Section 6.4 on systematic uncertainties).

4. Construct the likelihood function.

For a set of observed events $\{m_i\}$, the extended likelihood takes the form

$$\mathcal{L}(\mu, \nu) = \prod_{i=1}^N [\mu S f_S(m_i | \nu) + B f_B(m_i | \nu)] \times \exp[-(\mu S + B)] \times \prod_j \pi(\nu_j),$$

where:

- μ is the signal strength (SM corresponds to $\mu = 1$),
- ν are nuisance parameters representing systematic uncertainties,
- $\pi(\nu_j)$ are constraint terms (Gaussian/log-normal priors),
- S and B are expected signal and background yields.

This formalism is identical to the one introduced in Section ?? of Chapter 6.

5. Form a test statistic using the profile-likelihood ratio.

The LHC standard test statistic compares the restricted likelihood (background-only or fixed signal strength) to the full maximized likelihood:

$$q_\mu = -2 \ln \frac{\mathcal{L}(\mu, \hat{\nu}_\mu)}{\mathcal{L}(\hat{\mu}, \hat{\nu})}.$$

Here:

- $\hat{\mu}, \hat{\nu}$ maximize the full likelihood,
- $\hat{\nu}_\mu$ maximize the likelihood under fixed μ .

Profiling over nuisance parameters incorporates systematic uncertainties automatically and optimally — as discussed in detail in Section 6.4 (profile likelihoods and nuisance parameters).

The most important special case is:

$$q_0 = -2 \ln \frac{\mathcal{L}(0, \hat{\nu}_0)}{\mathcal{L}(\hat{\mu}, \hat{\nu})},$$

used to test for discovery.

6. Compute p -values or significance Z .

Under Wilks' theorem and asymptotic approximations (see Section 6.3), the local significance associated with q_0 is:

$$Z = \sqrt{q_0}, \quad p_0 = 1 - \Phi(Z),$$

where Φ is the standard normal CDF. This gives an approximate mapping between a small p -value and a number of “sigma” in the Gaussian sense—the language universally used in particle physics.

This is how ATLAS and CMS reported the significances of their Higgs peaks in the diphoton and 4ℓ invariant mass distributions.

7. Claim evidence or discovery.

By long-standing community convention:

$$\text{evidence: } Z \geq 3\sigma, \quad \text{discovery: } Z \geq 5\sigma.$$

Both experiments observed:

- a $\sim 4\sigma$ excess in $H \rightarrow \gamma\gamma$,
- a $\sim 6.5\sigma$ excess in $H \rightarrow ZZ^* \rightarrow 4\ell$,
- a combined significance above 7σ ,

leading to the July 4, 2012 discovery announcement.

The Higgs boson was observed as a *small resonant peak* on top of a large smooth background. Statistically, this is a classic problem of searching for a localized excess.

7.2 Reconstructing Decay Channels

Two channels provided the cleanest early evidence:

- **Diphoton channel:** $H \rightarrow \gamma\gamma$
 - small branching ratio ($\sim 2 \times 10^{-3}$),
 - superb mass resolution ($\sim 1\text{--}2\%$),
 - backgrounds: prompt diphotons, γ +jet, jet+jet fakes.
- **Four-lepton channel:** $H \rightarrow ZZ^* \rightarrow 4\ell$
 - even smaller branching ratio ($\sim 1.2 \times 10^{-4}$),
 - *golden channel* due to extremely low background,
 - backgrounds: continuum ZZ^* , Z +jets faking leptons.

Both channels reconstruct the invariant mass of the final-state particles, yielding a distribution in which a Higgs boson would appear as a narrow peak near 125 GeV.

7.3 Invariant Mass Reconstruction

Invariant mass reconstruction is the central experimental tool in resonance searches at the LHC. Because the Higgs boson is a narrow state, its presence is revealed as a localized excess in the invariant mass spectrum of its decay products.

This section explains in detail how invariant masses are constructed in the two golden Higgs channels and how detector resolution propagates into the observed mass peak.

7.3.1 Lorentz-Invariant Mass

For a system of final-state particles with four-momenta $p_i^\mu = (E_i, \vec{p}_i)$, the invariant mass is defined as

$$m^2 = \left(\sum_i p_i^\mu \right)^2 = \left(\sum_i E_i \right)^2 - \left| \sum_i \vec{p}_i \right|^2.$$

Key properties:

- Lorentz invariant under boosts and rotations,
- independent of the unknown longitudinal momentum of the initial partons,
- directly comparable to theoretical predictions.

This makes invariant mass the optimal observable for resonance searches.

7.3.2 Diphoton Invariant Mass Reconstruction

The Higgs decay $H \rightarrow \gamma\gamma$ produces two isolated, high-energy photons measured primarily in the electromagnetic calorimeter (ECAL).

Kinematic Formula

Given two photons with measured energies E_1, E_2 and opening angle θ_{12} , the invariant mass is

$$m_{\gamma\gamma} = \sqrt{2E_1E_2(1 - \cos\theta_{12})}.$$

This expression highlights two experimental inputs:

- **Photon energy measurement** (ECAL resolution),
- **Photon direction measurement** (calorimeter segmentation and vertex reconstruction).

Diphoton Geometry

Figure 7.1a illustrates the Feynman diagram for Higgs decays into diphoton: (Left) topquark loop (Right) $W^\pm$ boson loop. Figure 7.1b illustrates the diphoton decay geometry and the definition of the opening angle.

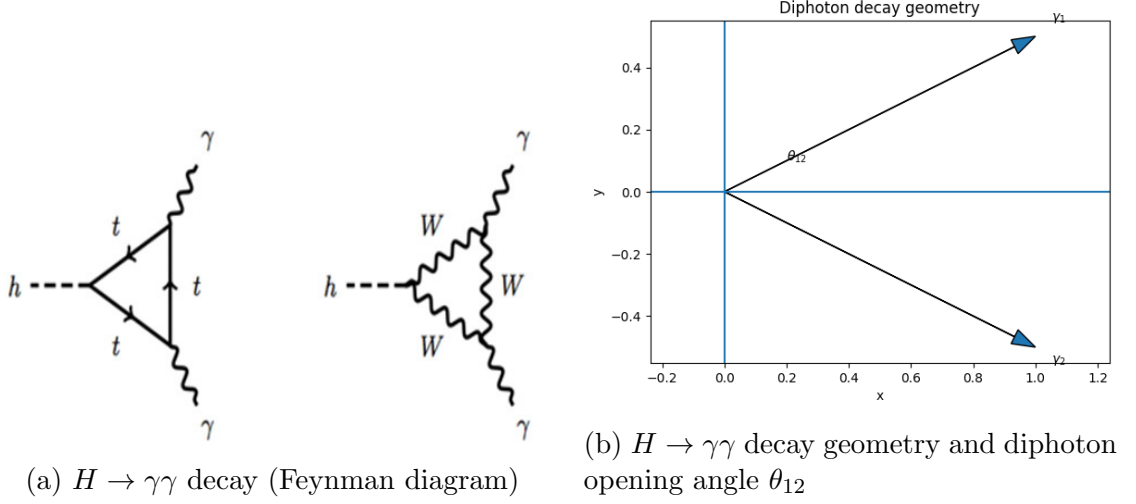


Figure 7.1: Schematic illustrations of Higgs boson decay topologies used for invariant-mass reconstruction. Left: diphoton decay channel. Right: diphoton opening angle and energy measurement.

Experimental Resolution

The diphoton mass resolution is dominated by:

- ECAL energy resolution ($\sigma_E/E \sim 1\%$),
- uncertainty in the primary vertex (affecting θ_{12}),
- material effects (photon conversions).

As a result, the observed Higgs peak has a width

$$\sigma_{m_{\gamma\gamma}} \approx 1.5\text{--}2 \text{ GeV},$$

much larger than the intrinsic Higgs width ($\sim 4 \text{ MeV}$).

7.3.3 Exercise: Numerical Mass Reconstruction

To illustrate invariant-mass reconstruction with a simplified numerical example based on the diphoton channel.

Given: Two photons are reconstructed with:

$$E_1 = 62 \text{ GeV}, \quad E_2 = 64 \text{ GeV}, \quad \theta_{12} = 0.62 \text{ rad.}$$

Task: Compute the reconstructed diphoton invariant mass and estimate the impact of energy resolution.

Solution: Using

$$m_{\gamma\gamma} = \sqrt{2E_1E_2(1 - \cos \theta_{12})},$$

we find:

$$m_{\gamma\gamma} = \sqrt{2 \times 62 \times 64 \times (1 - \cos 0.62)} \approx 124.7 \text{ GeV.}$$

This is consistent with a Higgs boson of mass $m_H \simeq 125 \text{ GeV}$.

Uncertainty propagation: Assuming a relative photon energy resolution

$$\frac{\sigma_E}{E} \approx 1\%,$$

the mass uncertainty is approximately

$$\frac{\sigma_m}{m} \approx \frac{1}{2} \left(\frac{\sigma_{E_1}}{E_1} + \frac{\sigma_{E_2}}{E_2} \right) \approx 1\%.$$

Thus,

$$\sigma_{m_{\gamma\gamma}} \sim 1.3 \text{ GeV},$$

consistent with experimental observations.

Interpretation: This simple calculation shows how:

- detector resolution dominates the observed peak width,
- the Higgs signal remains narrow relative to background variation,
- mass resolution directly controls discovery sensitivity.

7.3.4 Python Example: Toy Higgs Mass Spectrum

The following toy Monte Carlo simulates a Higgs-like resonance on top of a smooth background and reproduces the essential structure of the discovery plots.

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3
4 np.random.seed(42)
5
6 # Parameters
7 mH = 125.0
8 sigma = 1.5
9 Nsig = 200
10 Nbkg = 5000
11
12 # Generate signal and background
13 signal = np.random.normal(mH, sigma, Nsig)
14 background = np.random.exponential(scale=40, size=Nbkg) + 100
15
16 # Combine
17 known_mass = np.concatenate([signal, background])
18
19 # Plot
20 plt.hist(known_mass, bins=60, range=(100,160), histtype='step',
21          linewidth=1.5)
22 plt.xlabel(r"$m_{\gamma\gamma}$ [GeV]")
23 plt.ylabel("Events / bin")
24 plt.title("Toy Higgs Mass Reconstruction")
25 plt.show()

```

Code 7.1: Toy Monte Carlo Higgs mass spectrum

Pedagogical lessons:

- The Higgs signal appears as a localized excess,
- Background shape is smooth and featureless,
- The resulting histogram is the direct input to the likelihood defined in Section 7.1.

This example mirrors the statistical structure used in the full ATLAS and CMS analyses, albeit at a simplified level.

7.3.5 Four-Lepton Invariant Mass Reconstruction

The decay $H \rightarrow ZZ^* \rightarrow 4\ell$ produces four charged leptons (e or μ) measured with exceptional precision in the tracker and muon systems.

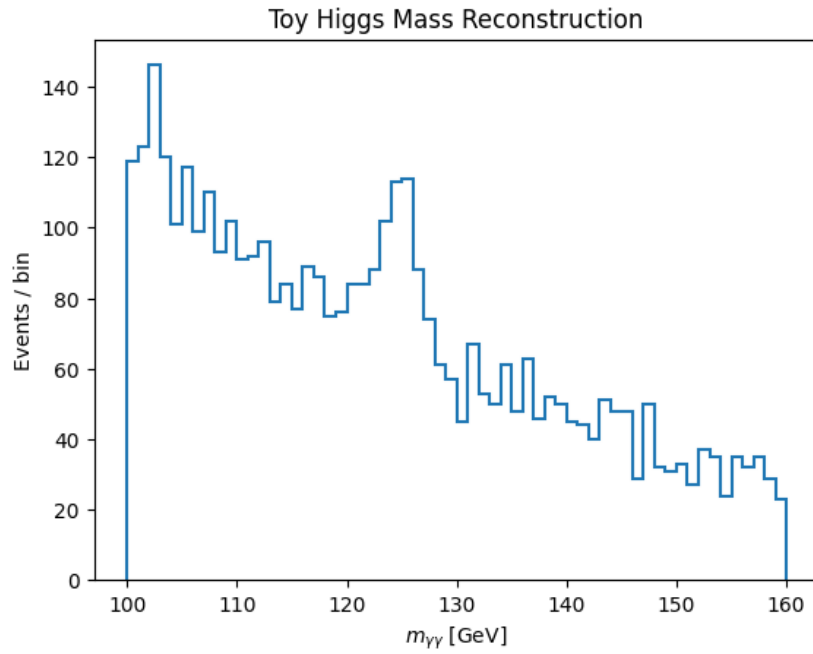


Figure 7.2: Toy Monte Carlo reconstruction of the diphoton invariant mass. A narrow Higgs signal at $m_H = 125$ GeV appears as a localized excess on top of a smooth background, illustrating the statistical challenge of resonance searches.

Decay Chain and Event Topology

The decay proceeds via:

$$H \rightarrow ZZ^* \rightarrow (\ell^+ \ell^-)(\ell^+ \ell^-),$$

where one Z boson may be off-shell.

Figure 7.3 shows the decay topology.

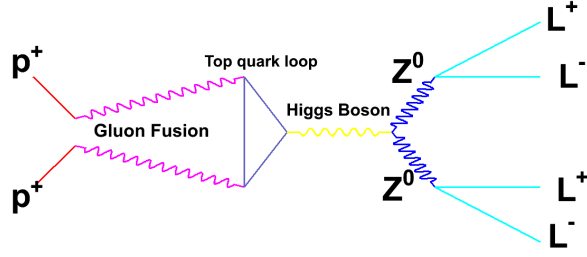


Figure 7.3: Decay chain for $H \rightarrow ZZ^* \rightarrow 4\ell$. Four isolated leptons fully reconstruct the Higgs four-momentum.

Mass Reconstruction

The invariant mass is computed as

$$m_{4\ell}^2 = (p_1^\mu + p_2^\mu + p_3^\mu + p_4^\mu)^2.$$

Because:

- lepton momenta are measured with $\mathcal{O}(0.1\%)$ precision,
- final state is fully reconstructed,

the mass resolution is excellent:

$$\sigma_{m_{4\ell}} \sim 1 \text{ GeV}.$$

This is why the 4ℓ channel is known as the *golden channel*.

7.3.6 Toy Monte Carlo Example: Mass Reconstruction

To illustrate invariant mass reconstruction, consider a toy Monte Carlo simulation of Higgs decays.

Toy model assumptions:

- True Higgs mass: $m_H = 125$ GeV,
- Gaussian detector resolution:

$$m_{\text{reco}} = m_H + \mathcal{N}(0, \sigma),$$

- $\sigma = 1.5$ GeV (diphoton-like),
- smooth exponential background.

Algorithm:

1. Generate N_S signal events from a Gaussian,
2. Generate N_B background events from an exponential,
3. Fill a mass histogram,
4. Observe the appearance of a resonant peak.

Pedagogical outcome:

- The Higgs signal appears as a small bump,
- Resolution controls peak width,
- Statistics control peak visibility,
- This distribution feeds directly into the likelihood described in Sections 7.1 and 6.2.

This toy model captures the essential statistical structure of the real ATLAS and CMS Higgs analyses.

7.3.7 Why Invariant Mass Is Optimal

Invariant mass reconstruction:

- maximizes signal-to-background separation,
- minimizes theoretical and experimental uncertainties,
- enables unbinned likelihood fits,
- directly supports hypothesis testing via profile likelihoods.

7.4 Signal and Background Modeling

7.4.1 Signal model

The Higgs signal in both channels appears as a narrow resonance. A common model is a Crystal Ball or double-sided Crystal Ball PDF:

$$S(m; m_H, \sigma, \alpha_L, \alpha_R, n_L, n_R).$$

For conceptual clarity, a simple Gaussian is often used in pedagogical examples:

$$S(m) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left[-\frac{(m - m_H)^2}{2\sigma^2}\right].$$

7.4.2 Background model

Backgrounds are smooth functions of mass.

Diphoton channel. Typical parameterizations:

$$B(m) = e^{a_0 + a_1 m + a_2 m^2}, \quad B(m) = (1 - m/\sqrt{s})^{p_1} m^{p_2}.$$

Four-lepton channel. The ZZ^* continuum is modeled with:

$$B(m) = (m - m_0)^\beta e^{-\gamma m},$$

plus small instrumental backgrounds.

7.5 Constructing the Likelihood Function

For an observed dataset of N candidate events with masses m_i , the extended unbinned likelihood for signal strength μ is:

$$\mathcal{L}(\mu, \nu) = \exp[-(\mu S + B)] \prod_{i=1}^N [\mu S f_S(m_i | \nu) + B f_B(m_i | \nu)],$$

where:

- μ is the **signal strength** ($\mu = 1$ is SM Higgs),
- S and B are expected signal and background yields,
- ν is the set of nuisance parameters (mass shifts, energy scale, background-shape parameters, efficiencies, luminosity),

- f_S and f_B are the signal and background PDFs.

ATLAS and CMS also use Gaussian or log-normal constraints:

$$\pi(\nu_j) = \exp\left[-\frac{(\nu_j - \nu_{j,0})^2}{2\sigma_j^2}\right].$$

The full likelihood is:

$$\mathcal{L}_{\text{full}} = \mathcal{L} \prod_j \pi(\nu_j).$$

7.6 Hypothesis Testing: Profile Likelihood Ratio

To test the background-only hypothesis $\mu = 0$, the test statistic is

$$q_0 = -2 \ln \frac{\mathcal{L}(\mu = 0, \hat{\nu}_0)}{\mathcal{L}(\hat{\mu}, \hat{\nu})}.$$

Where

- $\hat{\mu}, \hat{\nu}$ maximize the likelihood (unconstrained fit),
- $\hat{\nu}_0$ maximize likelihood under $\mu = 0$.

7.6.1 Wilks' theorem

For large samples,

$$q_0 \sim \chi_1^2.$$

The significance is

$$Z = \sqrt{q_0}.$$

Thus a Higgs discovery corresponds to:

$$Z \geq 5.$$

7.7 Application to $H \rightarrow \gamma\gamma$

This channel produced one of the two iconic discovery plots.

7.7.1 Signal extraction

A narrow bump at $m_{\gamma\gamma} \approx 125 \text{ GeV}$ is extracted by fitting the likelihood to the observed mass distribution.

7.7.2 Background shape

The diphoton background is steeply falling, but smooth. A flexible analytic function is used; bias studies are required to ensure:

background model does not introduce fake bumps.

7.7.3 Observed excess

Both ATLAS and CMS independently saw an excess with local significance

$$Z_{\gamma\gamma} \approx 4\sigma.$$

7.8 Application to $H \rightarrow ZZ^* \rightarrow 4\ell$

The four-lepton channel is nearly background-free.

7.8.1 Event selection

A typical selection requires:

- four isolated leptons,
- two opposite-sign, same-flavor lepton pairs,
- $m_{Z_1} \approx 91$ GeV, and m_{Z_2} soft,
- total four-lepton mass $m_{4\ell}$ in $[110, 140]$ GeV.

7.8.2 Mass peak

Because of excellent lepton resolution, the Higgs signal appears as a very sharp peak:

$$Z_{4\ell} \approx 6.5\sigma.$$

7.9 Combined Diphoton + Four-Lepton Significance

The two channels are statistically independent.

Combining their likelihoods:

$$\mathcal{L}_{\text{comb}} = \mathcal{L}_{\gamma\gamma} \mathcal{L}_{4\ell},$$

the total significance is

$$Z_{\text{combined}} \approx 7.0 \sigma,$$

which satisfies the discovery criterion.

This was the basis of the celebrated July 4th, 2012 announcement.

7.10 Angular Observables in $H \rightarrow ZZ^* \rightarrow 4\ell$

Beyond invariant mass, the four-lepton final state provides access to a rich set of angular observables that probe the *spin and parity* of the Higgs boson.

7.10.1 Decay Angles

The full kinematics of the process

$$H \rightarrow ZZ^* \rightarrow (\ell^+ \ell^-)(\ell^+ \ell^-)$$

is described by five angles and two invariant masses.

The most important angles are:

- θ_1 : decay angle of leptons from the first Z ,
- θ_2 : decay angle of leptons from the second Z ,
- ϕ : angle between the decay planes of the two Z bosons,
- Φ : angle between the production and decay planes.

7.10.2 Spin-Parity Sensitivity

Different spin-parity hypotheses predict distinct angular distributions:

- Scalar (0^+ , SM Higgs): isotropic distributions with characteristic correlations,
- Pseudoscalar (0^-): enhanced modulation in ϕ ,
- Spin-2: strongly anisotropic angular patterns.

The fully differential decay rate can be written schematically as:

$$\frac{d\Gamma}{d\Omega} = \sum_i c_i f_i(\theta_1, \theta_2, \phi, \Phi),$$

where the coefficients c_i depend on the Higgs quantum numbers.

7.10.3 Connection to Hypothesis Testing

Angular observables enter the likelihood as additional dimensions:

$$\mathcal{L}(\mu, J^P) = \prod_i [\mu f_S(m_i, \Omega_i | J^P) + f_B(m_i, \Omega_i)] .$$

ATLAS and CMS used such likelihoods to exclude non-scalar hypotheses at high significance shortly after the Higgs discovery.

Preview: These techniques will be revisited in Chapter 8 when discussing:

- multi-dimensional likelihoods,
- hypothesis testing between non-nested models,
- likelihood-ratio tests beyond Wilks' theorem.

7.11 Summary

In this chapter we introduced:

- scientific hypotheses and physical observables for Higgs searches,
- mass reconstruction in $\gamma\gamma$ and 4ℓ channels,
- signal and background modeling,
- profile-likelihood hypothesis testing,
- extraction of local significances,
- the contributions of the two golden channels.

Chapter 8

Hypothesis Testing in Particle Physics (Higgs Search II)

In Chapter 7, we introduced hypothesis testing through the Higgs boson searches in the $H \rightarrow \gamma\gamma$ and $H \rightarrow ZZ^* \rightarrow 4\ell$ channels. We focused on invariant-mass reconstruction, likelihood functions, and the computation of *local* significances. In this chapter, we extend that framework to the full statistical machinery required for a robust discovery claim:

- global versus local p -values,
- the Look–Elsewhere Effect (LEE),
- confidence levels and exclusion limits,
- asymptotic approximations versus Monte Carlo,
- statistical combinations of multiple channels.

These tools were essential for the Higgs discovery and remain the standard approach in modern LHC analyses.

8.1 Local vs Global Significance

8.1.1 Local Significance

The *local significance* quantifies the probability that a background-only model fluctuates to produce an excess *at a fixed mass hypothesis* m_H . At each tested mass value, the background-only hypothesis $\mu = 0$ is compared to the best-fit

signal-plus-background hypothesis using the profile-likelihood-ratio test statistic

$$q_0 = -2 \ln \frac{\mathcal{L}(\mu = 0, \hat{\nu}_0)}{\mathcal{L}(\hat{\mu}, \hat{\nu})},$$

where:

- $\hat{\mu}, \hat{\nu}$ maximize the likelihood without constraints,
- $\hat{\nu}_0$ maximize the likelihood under $\mu = 0$.

Under the asymptotic assumptions of Wilks' theorem (see Section 6.3), q_0 follows a χ^2 distribution with one degree of freedom, and the local significance is defined as

$$Z_{\text{local}} = \sqrt{q_0}.$$

From significance to p -value ; the corresponding local p -value is

$$p_0^{\text{local}} = 1 - \Phi(Z_{\text{local}}),$$

where $\Phi(Z)$ is the cumulative distribution function (CDF) of the standard normal distribution:

$$\Phi(Z) \equiv \int_{-\infty}^Z \frac{1}{\sqrt{2\pi}} e^{-t^2/2} dt.$$

Physically, p_0^{local} is the probability that a background-only experiment produces a statistical fluctuation at least as large as the one observed *at that specific mass value*. Because only upward fluctuations correspond to a signal-like excess, a one-sided test is used. By convention:

$$Z = 3 \Rightarrow \text{evidence}, \quad Z = 5 \Rightarrow \text{discovery}.$$

For reference:

Z_{local}	p_0^{local}
3σ	1.35×10^{-3}
4σ	3.2×10^{-5}
5σ	2.9×10^{-7}

Invariant-mass representation Figure 8.1 shows a toy invariant-mass distribution with a Higgs-like excess near $m_H = 125$ GeV. The quoted value of Z_{local} corresponds to the profile-likelihood-ratio test evaluated *at that fixed mass hypothesis*.

This is the quantity reported directly on the invariant-mass plots shown by ATLAS and CMS in the Higgs discovery papers.

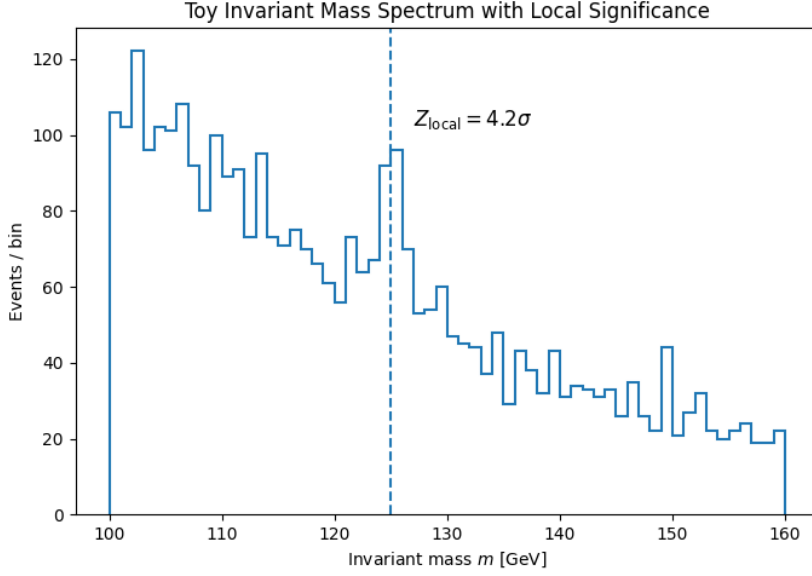


Figure 8.1: Toy invariant-mass spectrum illustrating the concept of local significance. The dashed vertical line marks the tested Higgs mass hypothesis. The quoted Z_{local} quantifies the probability for a background-only fluctuation to produce the observed excess at this mass.

Key limitation ; in a real search, the Higgs mass is *not known in advance*. The experiment scans over a wide mass range (e.g. 110 - 150 GeV), testing many correlated hypotheses.

The probability of observing *some* upward fluctuation anywhere in the search range is therefore larger than the probability of a fluctuation at a single, fixed mass. In other words, the local significance does *not* account for the fact that many mass hypotheses are tested simultaneously. Correcting for this multiple-testing effect leads to the *global significance*, discussed in the next section.

8.2 The Look-Elsewhere Effect (LEE)

8.2.1 Conceptual Origin

The Look-Elsewhere Effect arises whenever multiple hypotheses are tested simultaneously. In resonance searches, each tested mass point represents a different hypothesis.

If N independent mass points are tested, a rough approximation is:

$$p_{\text{global}} \approx 1 - (1 - p_{\text{local}})^N \approx N p_{\text{local}} \quad (p_{\text{local}} \ll 1).$$

The factor N is often called the *trials factor*.

8.2.2 Effective Number of Trials

Mass hypotheses are *correlated* because the detector resolution smooths the spectrum. The effective number of independent trials is therefore approximately:

$$N_{\text{eff}} \sim \frac{\text{search range}}{\text{mass resolution}}.$$

For example:

- search range: 110-150 GeV,
- resolution: ~ 1.5 GeV,
- $N_{\text{eff}} \sim 25$.

8.2.3 ATLAS and CMS Treatment of the Look-Elsewhere Effect

When searching for a new resonance with unknown mass, the test of the background-only hypothesis is not performed at a single mass value, but repeated over a *continuous range* of mass hypotheses. This multiple-testing problem gives rise to the *Look-Elsewhere Effect (LEE)*.

ATLAS and CMS account for the LEE by evaluating the probability that *anywhere in the scanned mass range* a background fluctuation could produce a signal-like excess at least as significant as the one observed.

Mass scanning procedure ; in a Higgs search, the invariant-mass spectrum is scanned over a predefined range (e.g. 110-150 GeV), using a sequence of closely spaced mass hypotheses m_H . At each mass point:

- the signal model is centered at m_H ,
- the profile-likelihood-ratio test statistic $q_0(m_H)$ is computed,
- the corresponding local significance $Z(m_H)$ is obtained.

This produces a *local significance curve* $Z(m_H)$, from which the largest observed excess

$$Z_{\text{obs}} = \max_{m_H} Z(m_H)$$

is identified.

Pseudo-experiments under the null hypothesis ; to quantify the probability of observing such an excess anywhere in the spectrum, ATLAS and CMS generate a large ensemble of *pseudo-experiments* assuming the background-only hypothesis H_0 .

Each pseudo-experiment consists of:

- generating a synthetic dataset drawn from the background model,
- applying the same mass scan procedure as for real data,
- computing $Z(m_H)$ across the full mass range,
- recording the maximum local significance $Z_{\max}$ in that dataset.

This procedure faithfully reproduces the trial factor associated with scanning over multiple mass hypotheses, including correlations between nearby mass points.

Distribution of the maximum significance ; repeating this process many times builds the sampling distribution

$$f(Z_{\max} \mid H_0),$$

which describes how often background-only experiments produce large signal-like excesses *anywhere* in the mass range. Because $Z_{\max}$ is an extreme-value statistic, its distribution is significantly shifted toward larger values compared to a single-point test.

Global p -value definition ; the *global p -value* is defined as

$$p_{\text{global}} = P(Z_{\max} \geq Z_{\text{obs}} \mid H_0),$$

i.e. the probability that a background-only experiment produces a maximum local significance equal to or larger than the one observed in data.

The global significance is then reported as

$$Z_{\text{global}} = \Phi^{-1}(1 - p_{\text{global}}),$$

where Φ^{-1} is the inverse cumulative distribution function of the standard normal distribution.

Impact on Higgs searches In the 2012 Higgs analyses:

- local significances near 125 GeV reached $Z_{\text{local}} \sim 5\text{--}6$,
- global significances were reduced by approximately $0.5\text{--}1\sigma$ after accounting for the LEE,
- the discovery claim remained robustly above 5σ .

The modest size of the LEE correction reflects the fact that:

- the search window was relatively narrow,
- the Higgs signal is localized and narrow,
- correlations between neighboring mass hypotheses reduce the effective number of independent trials.

Why pseudo-experiments are essential Although approximate analytic treatments of the LEE exist, ATLAS and CMS rely on pseudo-experiments because they:

- fully account for detector resolution and background shape,
- incorporate nuisance-parameter profiling automatically,
- correctly model correlations in the mass scan,
- provide a robust frequentist interpretation.

This methodology has become the standard for resonance searches at the LHC and beyond.

8.3 Confidence Levels and Exclusion Limits

8.3.1 Exclusion vs Discovery

Not observing a signal leads to *exclusion limits* rather than discovery. These are typically quoted at:

95% confidence level (CL).

The quantity of interest is the signal strength μ .

8.3.2 The CL_s Method

The LHC standard for exclusion is the CL_s method:

$$\text{CL}_s = \frac{p_{s+b}}{1 - p_b},$$

where:

- p_{s+b} is the p -value under signal+background,
- p_b is the p -value under background-only.

A signal hypothesis is excluded if:

$$\text{CL}_s < 0.05.$$

This construction avoids excluding signals to which the experiment has little sensitivity.

8.4 Asymptotic Formulae vs Monte Carlo

8.4.1 Asymptotic Approximations

Under mild conditions, the distribution of q_μ is known analytically (Cowan et al., EPJC 71 (2011) 1554):

$$q_\mu \sim \begin{cases} \chi_1^2 & \text{(two-sided)} \\ \frac{1}{2}\delta(q_\mu) + \frac{1}{2}\chi_1^2 & \text{(one-sided)} \end{cases}$$

These results allow fast evaluation of significances without expensive Monte Carlo.

8.4.2 When Monte Carlo Is Required

Monte Carlo pseudo-experiments are still essential when:

- sample sizes are small,
- likelihoods are non-Gaussian,
- nuisance parameters are highly non-linear,
- LEE must be evaluated accurately.

8.5 Combining Channels

8.5.1 Likelihood Combination

Independent channels are combined by multiplying likelihoods:

$$\mathcal{L}_{\text{comb}}(\mu, \nu) = \prod_c \mathcal{L}_c(\mu, \nu_c),$$

where correlated systematics (e.g. luminosity) share common nuisance parameters.

8.5.2 Higgs Discovery Combination

For the Higgs discovery:

- $H \rightarrow \gamma\gamma$: moderate significance, excellent mass resolution,
- $H \rightarrow ZZ^* \rightarrow 4\ell$: high purity, low statistics,
- other channels: WW , $b\bar{b}$, $\tau\tau$.

The combined significance exceeded 7σ , far above the discovery threshold.

8.6 Exercises

Problem 1: Local vs Global Significance

A resonance search scans 110–150 GeV with a mass resolution of 2 GeV. A local excess of $Z_{\text{local}} = 4.1\sigma$ is observed.

1. Estimate the effective number of trials.
2. Approximate the global p -value.
3. Convert it to a global significance.

Hint: use $p_{\text{global}} \approx N_{\text{eff}} p_{\text{local}}$.

Problem 2: CL_s Exclusion

You test a signal hypothesis with $\mu = 1$ and find:

$$p_{s+b} = 0.08, \quad p_b = 0.30.$$

1. Compute CL_s .
2. Is the signal excluded at 95% CL?

Problem 3: Channel Combination

Two independent channels yield:

$$Z_1 = 3.8\sigma, \quad Z_2 = 4.5\sigma.$$

1. Compute the combined significance assuming independence.
2. Compare to the quadrature approximation.

Hint: add log-likelihood ratios.

Problem 4: Mini Higgs Search and the Look-Elsewhere Effect

Physics setup Generate a dataset of reconstructed invariant masses m in the range

$$110 \text{ GeV} \leq m \leq 150 \text{ GeV}.$$

The dataset consists of:

- a smooth background distributed according to an exponential PDF,
- a possible narrow signal modeled as a Gaussian resonance.

Assume:

- signal width: $\sigma = 1.5 \text{ GeV}$,
- unknown signal mass m_H ,
- unknown signal strength $\mu \geq 0$.

Part A: Likelihood construction

1. Write down the unbinned likelihood

$$\mathcal{L}(\mu, m_H, \nu)$$

for signal plus background, including:

- signal normalization μ ,
 - background normalization,
 - any necessary nuisance parameters.
2. Explain clearly which parameters are:
 - parameters of interest,
 - nuisance parameters.

Part B: Local significance scan

1. Scan the mass range in steps of $\Delta m = 0.5$ GeV.
2. At each mass hypothesis m_H , compute the profile-likelihood-ratio test statistic

$$q_0(m_H).$$

3. Convert $q_0(m_H)$ into a local significance

$$Z_{\text{local}}(m_H).$$

4. Produce a plot of Z_{local} versus m_H .

Hint: you may use asymptotic formulae rather than pseudo-experiments for the local significance.

Part C: Observed excess

1. Identify the mass m_H^{best} at which the maximum local significance occurs.
2. Quote the corresponding $Z_{\text{local}}^{\text{max}}$.
3. Interpret this value physically.

Part D: Look-Elsewhere Effect

1. Estimate the effective number of trials

$$N_{\text{eff}} \approx \frac{\text{search range}}{\text{mass resolution}}.$$

2. Using this estimate, compute an approximate global p -value:

$$p_{\text{global}} \approx N_{\text{eff}} p_{\text{local}}^{\text{min}}.$$

3. Convert this into a global significance Z_{global} .

Part E: Interpretation

1. Compare Z_{local} and Z_{global} .
2. Discuss why the global significance is smaller.
3. State whether your result would qualify as:
 - evidence,
 - discovery,
 - or neither.

Deliverables Submit:

- a short written report (3-5 pages),
- plots of the invariant-mass spectrum and $Z(m_H)$ scan,
- clear explanations of statistical assumptions.

Problem 5: Pseudo-Experiments and Extreme-Value Statistics

In this problem you will study the Look–Elsewhere Effect using Monte Carlo pseudo-experiments.

1. Generate many background-only pseudo-experiments.
2. For each pseudo-experiment:
 - perform a mass scan,
 - compute $Z_{\text{local}}(m_H)$,
 - record Z_{max} .
3. Build the distribution $f(Z_{\text{max}} \mid H_0)$.
4. Compute the global p -value for an observed $Z_{\text{local}}^{\text{max}}$.

Discuss how this Monte Carlo result compares to the analytic trials-factor approximation used in Problem 4.

This problem mirrors the actual ATLAS and CMS discovery methodology.

8.7 Summary

In this chapter we introduced:

- local and global significances,
- the Look–Elsewhere Effect,
- confidence levels and CL_s ,
- asymptotic formulae and pseudo-experiments,
- statistical combinations of Higgs channels.

